

RTL Design Sherpa

APB GPIO Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Gpio Mas Index....15
2 APB GPIO - Overview....16
2.1 Overview....16
2.1.1 Features....16
2.1.2 Applications....16
2.1.3 Figure 1.1: APB GPIO Block Diagram....17
2.2 Parameters....17
2.2.1 Register Summary....18
2.3 Navigation....19
3 APB GPIO - Architecture....20
3.1 Overview....20
3.1.1 Figure 1.2: APB GPIO Top-Level Architecture....20
3.1.2 Figure 1.3: APB GPIO Module Hierarchy....20
3.2 Functional Description....21
3.2.1 Data Flow....21
3.2.2 Clock Domains....23
3.3 Parameters....24
3.4 Design Notes....25
3.4.1 Resource Estimates....25
3.5 Navigation....25
4 APB GPIO - Clocks and Reset....26
4.1 Overview....26

4.1.1 Clock Signals....26
4.1.2 Reset Signals....26
4.2 Functional Description....27
4.2.1 Reset Behavior....27
4.2.2 Clock Domain Crossing....27
4.2.3 Input Synchronization....28
4.3 Timing....28
4.3.1 Timing Constraints....28
4.4 Navigation....29
5 APB GPIO - Acronyms and Terminology....30
5.1 Overview....30
5.1.1 Protocol Acronyms....30
5.1.2 Signal Acronyms....30
5.1.3 GPIO-Specific Terms....30
5.1.4 Interrupt Terms....31
5.1.5 Register Terms....31
5.2 Navigation....31
6 APB GPIO - References....32
6.1 References....32
6.1.1 Internal Documentation....32
6.1.2 External References....32
6.1.3 Design References....32
6.2 Navigation....33
7 APB GPIO - Block Descriptions Overview....34

7.1 Overview....34
7.1.1 Figure 2.1: APB GPIO Module Hierarchy....34
7.1.2 Block Summary....34
7.1.3 In This Chapter....34
7.2 Navigation....35
8 APB GPIO - APB Interface Block....36
8.1 Overview....36
8.1.1 Block Diagram....36
8.2 Ports....36
8.2.1 APB Slave Interface....36
8.3 Functional Description....37
8.3.1 Read Transaction....37
8.3.2 Write Transaction....37
8.4 Timing....37
8.4.1 Figure 2.2: APB Access Timing....38
8.5 Design Notes....38
8.6 Navigation....38
9 APB GPIO - Register File Block....39
9.1 Overview....39
9.1.1 Block Diagram....39
9.2 Ports....39
9.2.1 Hardware-to-Software (hw2reg)....39
9.2.2 Software-to-Hardware (reg2hw)....40
9.3 Functional Description....40

9.3.1 Byte Enable Support....40
9.3.2 Access Types....41
9.4 Design Notes....41
9.5 Navigation....41
10 APB GPIO - GPIO Core Block....42
10.1 Overview....42
10.1.1 Block Diagram....42
10.2 Ports....42
10.3 Functional Description....43
10.3.1 Input Path....43
10.3.2 Output Path....43
10.3.3 Direction Control....44
10.4 Design Notes....44
10.5 Navigation....45
11 APB GPIO - Interrupt Controller Block....46
11.1 Overview....46
11.1.1 Figure 2.5: Interrupt Controller Block Diagram....46
11.2 Functional Description....46
11.2.1 Interrupt Modes....46
11.2.2 Edge Detection Logic....48
11.2.3 Interrupt Status....48
11.2.4 Interrupt Enable....48
11.2.5 Aggregate IRQ Output....49
11.2.6 Interrupt Handling Flow....49

11.3 Design Notes....49
11.4 Navigation....49
12 APB GPIO - CDC Logic Block....51
12.1 Overview....51
12.1.1 Figure 2.7: CDC Logic Block Diagram....51
12.2 Parameters....52
12.2.1 CDC_ENABLE....52
12.3 Functional Description....52
12.3.1 Clock Domains....52
12.3.2 CDC Implementation....53
12.3.3 The irq Output....53
12.3.4 Coherency....54
12.3.5 Reset Synchronization....54
12.4 Timing....54
12.4.1 Latency....54
12.5 Navigation....54
13 APB GPIO - Interfaces Overview....56
13.1 Overview....56
13.1.1 Interface Summary Diagram....56
13.1.2 In This Chapter....57
13.2 Navigation....57
14 APB GPIO - APB Slave Interface....58
14.1 Ports....58
14.1.1 APB Slave Signals....58

14.2 Functional Description..... 58
14.2.1 Protocol Compliance..... 58
14.2.2 Address Decoding..... 59
14.2.3 Byte Strobes..... 60
14.2.4 Error Handling..... 60
14.3 Timing..... 60
14.3.1 Access Timing..... 60
14.3.2 Figure 3.1: APB Access Timing (CDC_ENABLE=0)..... 60
14.4 Navigation..... 61
15 APB GPIO - GPIO Pin Interface..... 62
15.1 Ports..... 62
15.1.1 GPIO Signals..... 62
15.2 Functional Description..... 62
15.2.1 Output Mode (direction[i] = 1)..... 62
15.2.2 Input Mode (direction[i] = 0)..... 63
15.2.3 Input Synchronization..... 63
15.2.4 Electrical Considerations..... 64
15.3 Timing..... 64
15.3.1 Output Path..... 64
15.3.2 Input Path..... 64
15.4 Navigation..... 64
16 APB GPIO - Interrupt Interface..... 65
16.1 Ports..... 65
16.2 Functional Description..... 65

16.2.1 Interrupt Generation..... 65
16.2.2 Per-Pin Configuration..... 65
16.2.3 Interrupt Modes..... 65
16.2.4 Interrupt Handling..... 67
16.3 Timing..... 68
16.3.1 Edge-Triggered Latency..... 68
16.3.2 Level-Sensitive Latency..... 69
16.4 Design Notes..... 69
16.4.1 Connection Guidelines..... 69
16.5 Navigation..... 70
17 APB GPIO - System Interface..... 71
17.1 Ports..... 71
17.1.1 pclk - APB Clock..... 71
17.1.2 gpio_clk - GPIO Clock..... 71
17.1.3 presetn - APB Reset..... 71
17.1.4 gpio_rstn - GPIO Reset..... 71
17.2 Functional Description..... 72
17.2.1 Clock Configurations..... 72
17.2.2 Reset Sequence..... 73
17.2.3 Figure 3.2: Power-On Reset Sequence..... 74
17.3 Timing..... 74
17.3.1 Clock Relationship..... 74
17.3.2 Reset Recovery..... 74
17.4 Navigation..... 74

18 APB GPIO - Programming Model Overview....76
18.1 Overview....76
18.1.1 In This Chapter....77
18.2 Navigation....77
19 APB GPIO - Basic Operations....78
19.1 Waveforms....78
19.1.1 Direction Configuration....78
19.1.2 Output Write....78
19.1.3 Input Read....79
19.1.4 Input Synchronization....80
19.1.5 Atomic Operations....81
19.2 Usage Example....82
19.2.1 Initialization....82
19.2.2 Output Operations....82
19.2.3 Input Operations....83
19.2.4 Mixed I/O Configuration....83
19.2.5 Output Enable Behavior....83
19.3 Navigation....84
20 APB GPIO - Interrupt Configuration....85
20.1 Waveforms....85
20.1.1 Rising Edge Interrupt....85
20.1.2 Falling Edge Interrupt....85
20.1.3 Level-Sensitive Interrupt....86
20.1.4 Interrupt Clear (W1C)....86

20.2 Functional Description..... 87
20.2.1 Interrupt Configuration Table..... 87
20.3 Usage Example..... 88
20.3.1 Interrupt Setup..... 88
20.3.2 Complete Setup Examples..... 88
20.3.3 Interrupt Handling..... 89
20.3.4 Level-Sensitive Considerations..... 90
20.4 Navigation..... 90
21 APB GPIO - Programming Examples..... 91
21.1 Usage Example..... 91
21.1.1 LED Control..... 91
21.1.2 Button Input..... 92
21.1.3 DIP Switch Reading..... 93
21.1.4 Parallel Data Interface..... 93
21.1.5 PWM-Style Output..... 94
21.1.6 Wake-On-Change..... 94
21.2 Navigation..... 95
22 APB GPIO - Software Considerations..... 96
22.1 Design Notes..... 96
22.1.1 Performance..... 96
22.1.2 Synchronization..... 97
22.1.3 Interrupt Best Practices..... 97
22.1.4 Error Handling..... 98
22.1.5 Debug Tips..... 98

22.2 Navigation....99
23 APB GPIO - Register Map....100
23.1 Overview....100
23.2 GPIO_CONTROL (0x000)....101
23.3 GPIO_DIRECTION (0x004)....102
23.4 GPIO_OUTPUT (0x008)....102
23.5 GPIO_INPUT (0x00C)....102
23.6 GPIO_INT_ENABLE (0x010)....103
23.7 GPIO_INT_TYPE (0x014)....103
23.8 GPIO_INT_POLARITY (0x018)....103
23.9 GPIO_INT_BOTH (0x01C)....104
23.10 GPIO_INT_STATUS (0x020)....104
23.11 GPIO_RAW_INT (0x024)....105
23.12 GPIO_OUTPUT_SET (0x028)....105
23.13 GPIO_OUTPUT_CLR (0x02C)....105
23.14 GPIO_OUTPUT_TGL (0x030)....106
23.15 Design Notes....106
23.15.1 Atomic-Register Write Semantics (SET/CLR/TGL)....106
23.15.2 Address Calculation....107
23.16 Navigation....107

List of Figures

Figure 1.1: APB GPIO Block Diagram....17
Figure 1.2: APB GPIO Top-Level Architecture....20
Figure 1.3: APB GPIO Module Hierarchy....20
Figure 1.4: Write Transaction Flow....21
Figure 1.5: Read Transaction Flow....22
Figure 1.6: Interrupt Flow....22
Figure 1.7: Synchronous Mode Clock Domains....23
Figure 1.8: Asynchronous Mode Clock Domains....24
Figure 2.1: APB GPIO Module Hierarchy....34
Figure 2.2: APB Access Timing....38
Figure 2.5: Interrupt Controller Block Diagram....46
Figure 2.6: Edge Detection Logic....48
Figure 2.7: CDC Logic Block Diagram....51
Figure 2.8: Single Clock Domain (CDC Disabled)....52
Figure 2.9: Dual Clock Domain (CDC Enabled)....53
Figure 3.1: APB Access Timing (CDC_ENABLE=0)....60
Figure 3.2: Power-On Reset Sequence....74

List of Tables

Table 1: Table 1.1: GPIO Parameters....24
Table 2: Table 1.2: Resource Estimates....25
Table 3: Table 2.1: Block Summary....34
Table 4: Table 2.5: Edge-Triggered Modes....47
Table 5: Table 2.6: Level-Sensitive Modes....47
Table 6: Table 2.7: CDC Enable Parameter....52
Table 7: Table 2.8: CDC Latency....54

List of Waveforms

No waveforms in this document.

1 Gpio Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 APB GPIO - Overview

2.1 Overview

The APB GPIO controller is a 32-bit general-purpose I/O block on the APB bus. Software gets per-bit direction control, per-bit interrupt configuration, and atomic output updates; the system gets a single interrupt line and a clean tri-state interface to the pads. Nothing exotic — which is exactly what you want from a GPIO.

2.1.1 Features

2.1.1.1 Core Functionality

- 32-bit bidirectional GPIO port
- Per-bit direction control (input/output)
- Per-bit output enable control
- Input synchronization for metastability protection

2.1.1.2 Interrupt Capabilities

- Per-bit interrupt enable
- Edge-triggered interrupts (rising, falling, or both)
- Level-triggered interrupts (high or low)
- Combined interrupt output (OR of all enabled sources)
- Write-1-to-clear interrupt status

2.1.1.3 Atomic Operations

- Atomic set (OR with mask)
- Atomic clear (AND with inverted mask)
- Atomic toggle (XOR with mask)
- No read-modify-write race conditions

2.1.1.4 Clock Domain Crossing

- Optional CDC support via `CDC_ENABLE` parameter
- Separate GPIO clock domain for async I/O
- Multi-stage input synchronization

2.1.2 Applications

2.1.2.1 Typical Use Cases

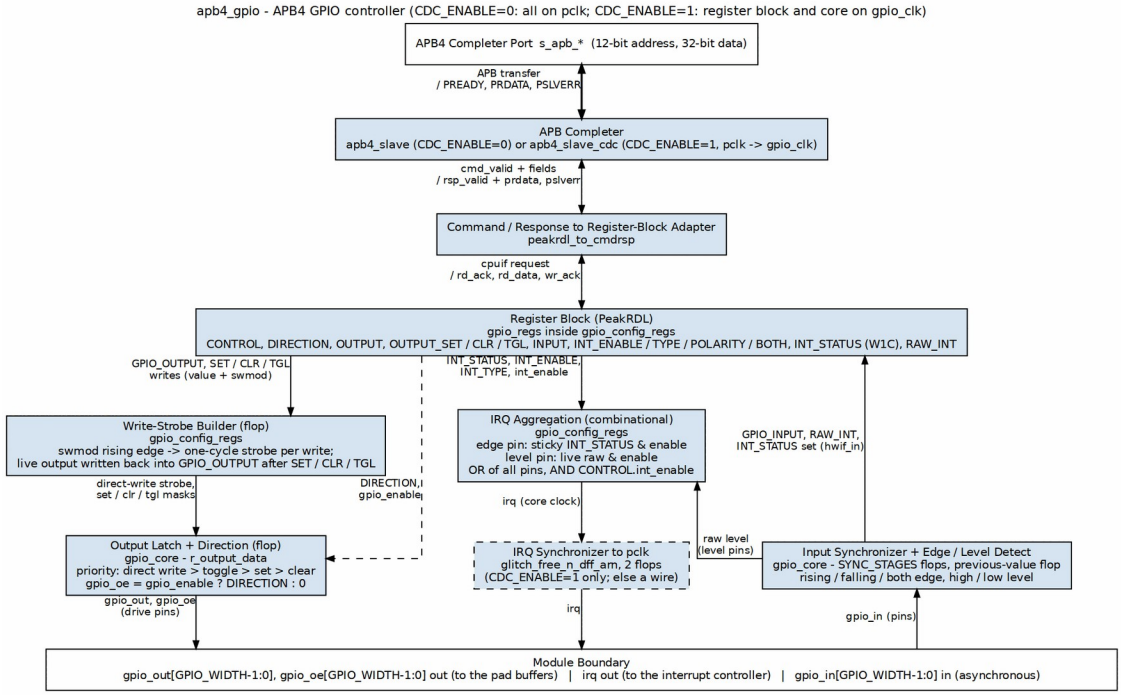
- LED control and status indication
- Push-button and switch inputs

- External device reset control
- Interrupt generation from external events
- Bit-banged serial protocols (I2C, SPI fallback)
- Debug signals and test points

2.1.2.2 System Integration

- Memory-mapped APB peripheral
- Single interrupt line to CPU/interrupt controller
- Direct connection to FPGA I/O pads via IOBUFs
- Compatible with standard GPIO software drivers

2.1.3 Figure 1.1: APB GPIO Block Diagram



APB GPIO Block Diagram

2.2 Parameters

The headline numbers. GPIO width and synchronizer depth are parameters, so treat the 32-bit default as exactly that — a default.

Parameter	Value
GPIO Width	32 bits (configurable)
APB Data Width	32 bits

Parameter	Value
APB Address Width	12 bits (4KB)
Sync Stages	2 (configurable)
CDC Support	Optional

2.2.1 Register Summary

The full map with field-level detail is in Chapter 5. Here's the quick reference:

Offset	Name	Access	Reset	Description
0x000	GPIO_CONTROL	RW	0x00000001	Global enable and interrupt enable
0x004	GPIO_DIRECTION	RW	0x00000000	Per-bit direction (1=output)
0x008	GPIO_OUTPUT	RW	0x00000000	Output data value
0x00C	GPIO_INPUT	RO	-	Input data (read-only)
0x010	GPIO_INT_ENABLE	RW	0x00000000	Per-bit interrupt enable
0x014	GPIO_INT_TYPE	RW	0x00000000	Interrupt type (1=level, 0=edge)
0x018	GPIO_INT_POLARITY	RW	0xFFFFFFFF	Polarity (1=high/rising)
0x01C	GPIO_INT_BOTH	RW	0x00000000	Both-edge enable
0x020	GPIO_INT_STATUS	W1C	0x00000000	Interrupt status (W1C)
0x024	GPIO_RAW_INT	RO	0 (live)	Raw interrupt (pre-mask)
0x028	GPIO_OUTPUT_SET	WO	0x00000000	Atomic set
0x02C	GPIO_OUTPUT_CL	WO	0x00000000	Atomic clear

Offset	Name	Access	Reset	Description
	R			
0x030	GPIO_OUTPUT_TGL	WO	0x00000000	Atomic toggle

2.3 Navigation

Next: 02_architecture.md - High-level architecture

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

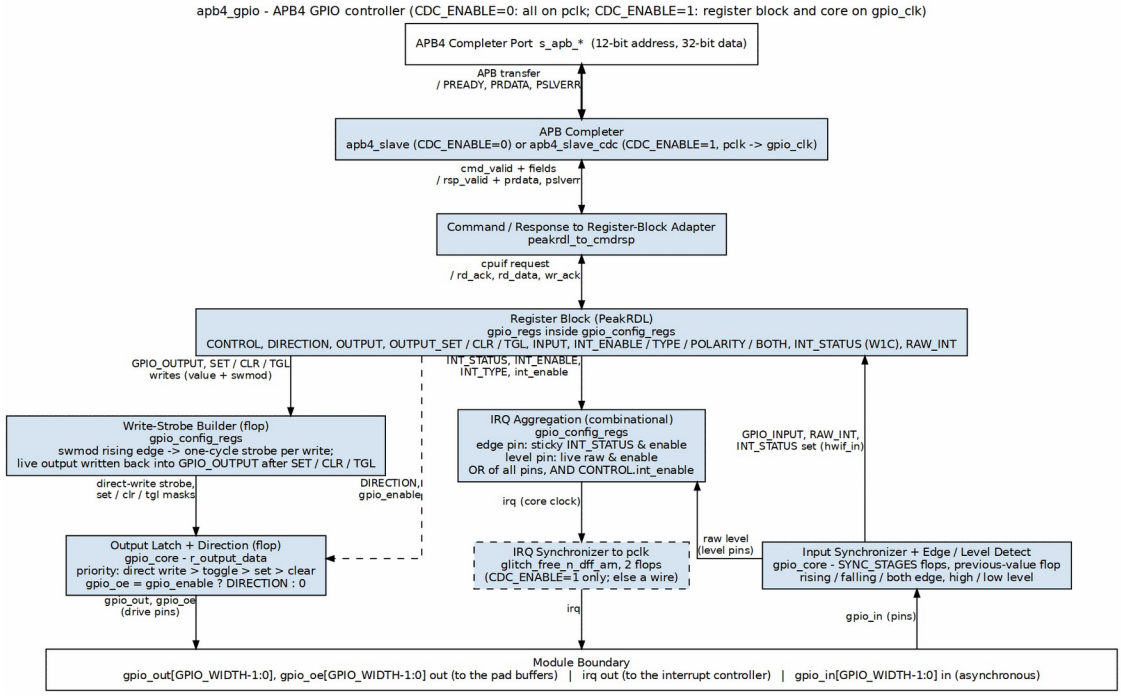
---

3 APB GPIO - Architecture

3.1 Overview

Two views of the same design: the top-level block diagram, and the module hierarchy underneath it.

3.1.1 Figure 1.2: APB GPIO Top-Level Architecture



APB GPIO Architecture

3.1.2 Figure 1.3: APB GPIO Module Hierarchy

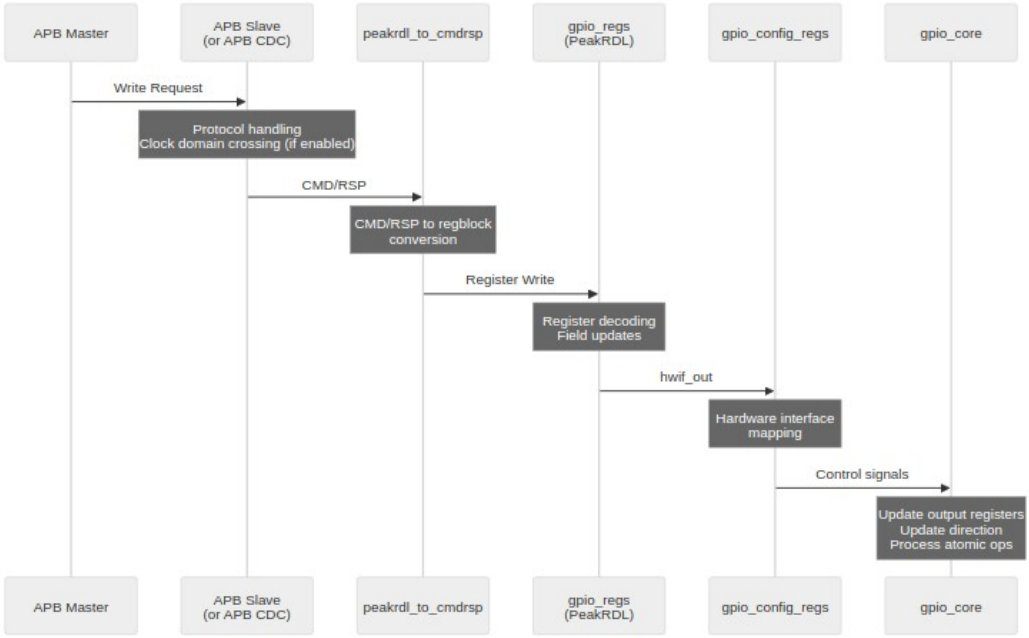
```
apb4_gpio                       top level
├── apb4_slave                  APB completer (CDC_ENABLE=0)
│   or apb4_slave_cdc           APB completer + crossing (CDC_ENABLE=1)
├── peakrdl_to_cmdrsp           cmd/rsp to register-block adapter
├── gpio_config_regs            write strobes, W1C merge, irq
│   ├── gpio_regs               PeakRDL register block
│   └── gpio_core               sync, output latch, edge/level
└── glitch_free_n_dff_arn       irq sync to pclk (CDC_ENABLE=1)
```

3.2 Functional Description

3.2.1 Data Flow

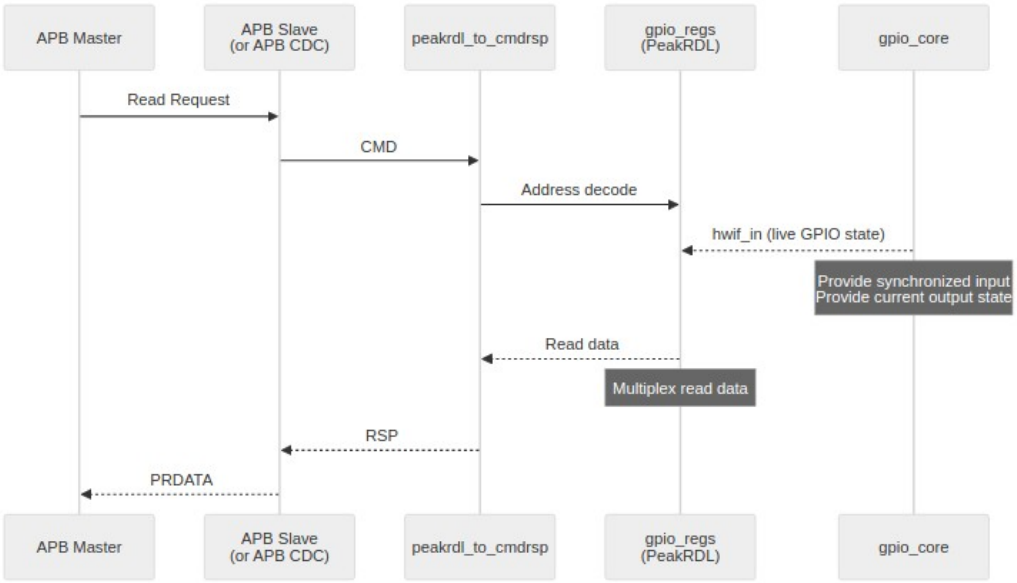
Three flows matter: APB writes, APB reads, and interrupts.

3.2.1.1 Figure 1.4: Write Transaction Flow



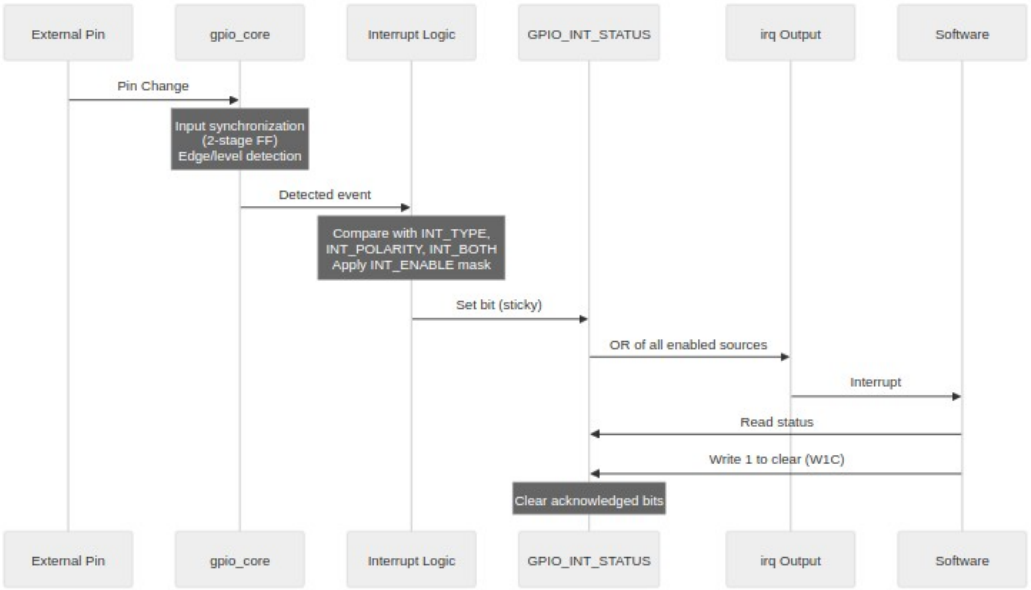
Write Transaction Flow

3.2.1.2 Figure 1.5: Read Transaction Flow



Read Transaction Flow

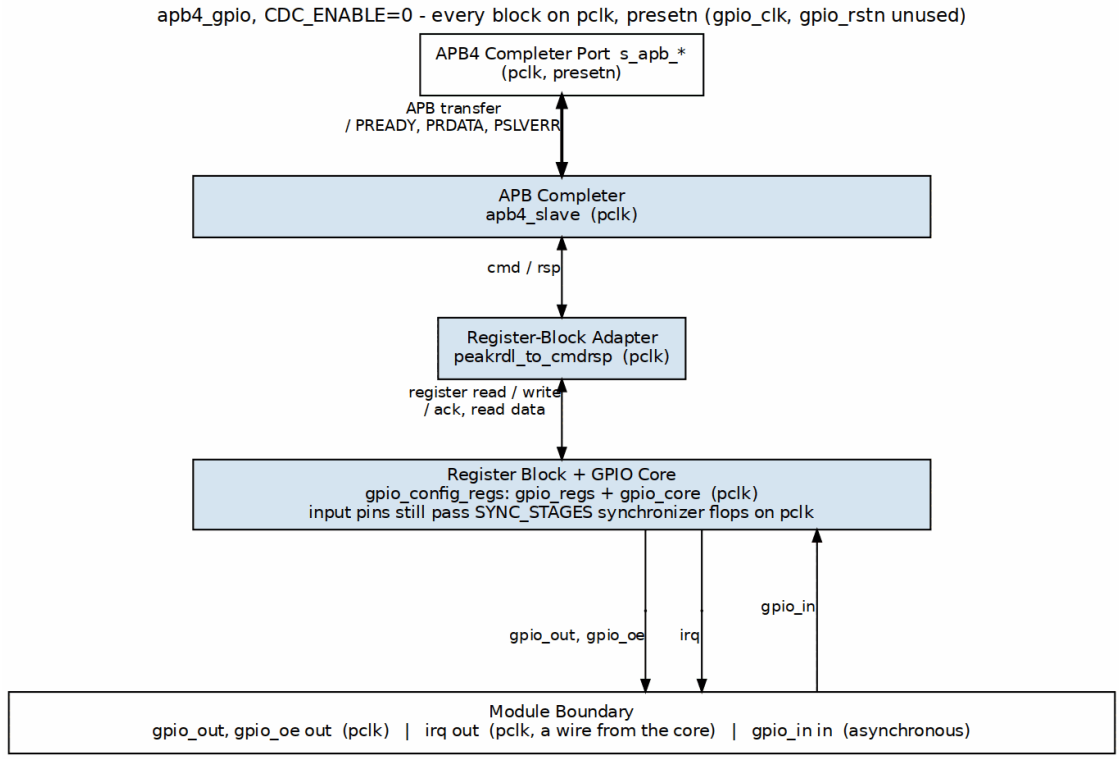
3.2.1.3 Figure 1.6: Interrupt Flow



Interrupt Flow

3.2.2 Clock Domains

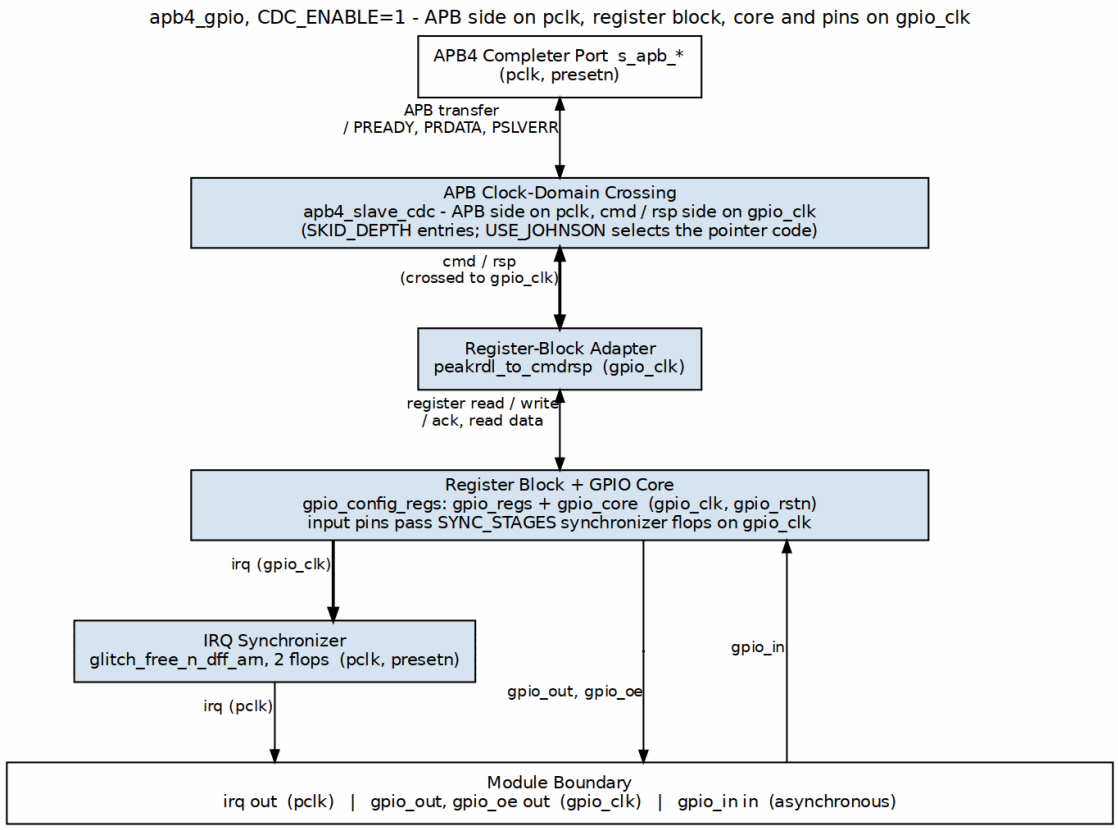
3.2.2.1 Figure 1.7: Synchronous Mode Clock Domains



Synchronous Mode

With CDC_ENABLE = 0, everything runs on the APB clock (pclk). Input synchronization stays active on the external GPIO pins — metastability doesn't care that your bus fabric is synchronous.

3.2.2.2 Figure 1.8: Asynchronous Mode Clock Domains



Asynchronous Mode

With CDC_ENABLE = 1, the APB clock domain handles protocol conversion while the GPIO clock domain owns the registers and I/O. The crossing is the pair of asynchronous FIFOs inside `apb4_slave_cdc` (command in, response out), plus a 2-flop synchronizer that brings `irq` back to pclk.

3.3 Parameters

Table 1.1: GPIO Parameters

Parameter	Type	Default	Description
`GPIO_WIDTH`	int	32	Number of GPIO pins
`SYNC_STAGES`	int	2	Input synchronizer stages
`CDC_ENABLE`	int	0	Enable clock domain crossing
`SKID_DEPTH`	int	2	CDC skid buffer depth

Parameter	Type	Default	Description
USE_JOHNSON	int	0	CDC counter encoding (forwarded to apb4_slave_cdc)

3.4 Design Notes

3.4.1 Resource Estimates

Rough synthesis numbers for planning purposes:

Table 1.2: Resource Estimates

Component	Flip-Flops	LUTs
gpio_core	~200	~300
gpio_regs	~400	~200
gpio_config_regs	~50	~100
apb4_slave (no CDC)	~20	~50
apb4_slave_cdc	~100	~150
Total (no CDC)	~670	~650
Total (with CDC)	~750	~750

3.5 Navigation

Next: 03_clocks_and_reset.md - Clock and reset behavior

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 APB GPIO - Clocks and Reset

4.1 Overview

Two clocks, two resets, and one synchronizer chain you can't turn off. Here's what each piece does.

4.1.1 Clock Signals

4.1.1.1 pclk (APB Clock)

- **Purpose:** Primary APB bus clock
- **Usage:** APB protocol, register access
- **Typical Frequency:** 50-200 MHz

4.1.1.2 gpio_clk (GPIO Clock)

- **Purpose:** Optional separate GPIO clock domain
- **Usage:** Only when `CDC_ENABLE=1`
- **Relationship:** Can be asynchronous to pclk

4.1.2 Reset Signals

4.1.2.1 presetn (APB Reset)

- **Type:** Active-low asynchronous reset (exception: the PeakRDL-generated register file resets SYNCHRONOUSLY – see the note in ch03_interfaces/04_system.md)
- **Scope:** APB interface logic
- **Behavior:** Resets APB state machine, clears pending transactions

4.1.2.2 gpio_rstn (GPIO Reset)

- **Type:** Active-low asynchronous reset
- **Scope:** GPIO core logic
- **Usage:** Only when `CDC_ENABLE=1`
- **Behavior:** Resets GPIO outputs, interrupt state

4.2 Functional Description

4.2.1 Reset Behavior

4.2.1.1 Register Reset Values

Register	Reset Value	Notes
GPIO_CONTROL	0x00000001	GPIO enabled; global interrupt enable clear
GPIO_DIRECTION	0x00000000	All inputs
GPIO_OUTPUT	0x00000000	Outputs low
GPIO_INT_ENABLE	0x00000000	No interrupts
GPIO_INT_TYPE	0x00000000	Edge mode
GPIO_INT_POLARITY	0xFFFFFFFF	Rising/high
GPIO_INT_BOTH	0x00000000	Single edge
GPIO_INT_STATUS	0x00000000	No pending
GPIO_RAW_INT	0 (live RO readback; depends on pins after reset)	No events
GPIO_OUTPUT_SET/CLR/TGL	0x00000000	No pending atomic op

GPIO_CONTROL.ENABLE resets to 1, but don't let that fool you into thinking pins drive out of reset: GPIO_DIRECTION resets to 0, so every pin is an input and high-Z until software programs directions. The `irq` output stays deasserted until software sets GPIO_CONTROL.INT_ENABLE (bit 1), which resets to 0.

4.2.1.2 Output Pin Behavior During Reset

During reset: - `gpio_out[31:0]` = 0 - `gpio_oe[31:0]` = 0 (all high-Z) - `irq` = 0

4.2.2 Clock Domain Crossing

4.2.2.1 When CDC_ENABLE = 0

- All logic runs on `pclk`
- `gpio_clk` input is ignored
- Connect `gpio_clk = pclk` for clean design

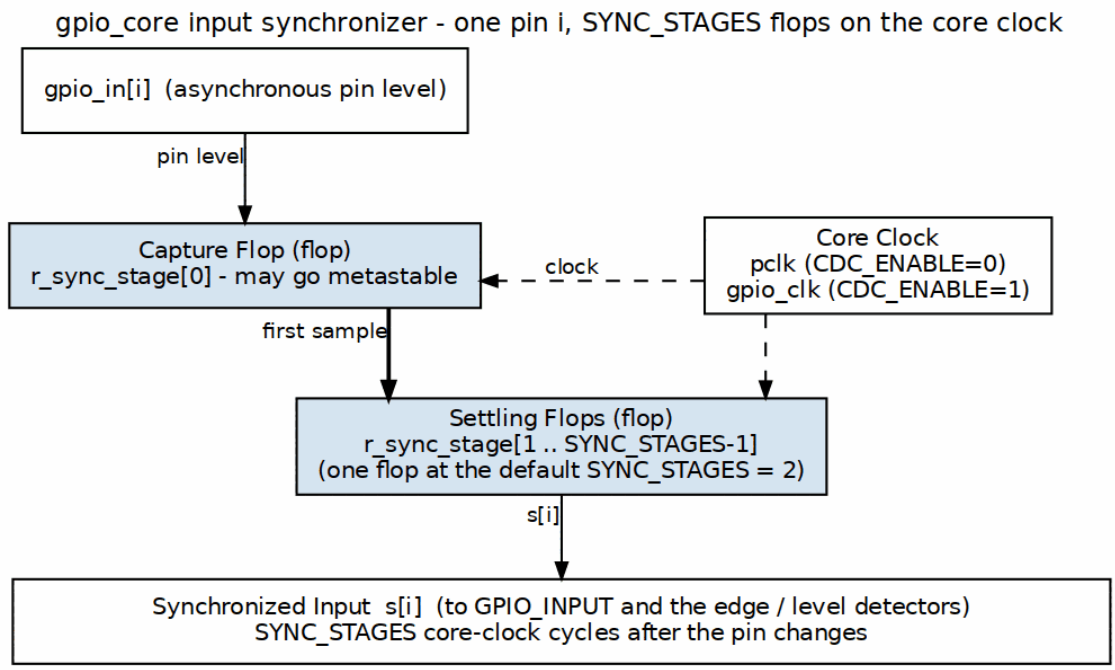
4.2.2.2 When CDC_ENABLE = 1

- APB interface uses `pclk`

- GPIO core uses `gpio_clk`
- `apb4_slave_cdc`'s asynchronous command and response FIFOs carry the APB traffic across; `irq` returns to pclk through a 2-flop synchronizer
- Both resets must be asserted together at power-on

4.2.3 Input Synchronization

GPIO inputs are always synchronized, CDC setting or not:



Input Synchronizer

- SYNC_STAGES parameter controls depth (default: 2)
- Prevents metastability from external signal transitions
- Adds latency equal to SYNC_STAGES clock cycles

4.3 Timing

4.3.1 Timing Constraints

4.3.1.1 Synchronous Mode

- Standard single-clock timing
- All paths constrained to pclk

4.3.1.2 Asynchronous Mode

- Set false_path between pclk and gpio_clk domains
- Set max_delay for CDC paths
- Synchronizer FFs should have ASYNC_REG attribute

4.4 Navigation

Next: 04_acronyms.md - Acronyms and terminology

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 APB GPIO - Acronyms and Terminology

5.1 Overview

The alphabet soup, collected in one place so the rest of the document doesn't have to stop and define itself.

5.1.1 Protocol Acronyms

Acronym	Full Name	Description
APB	Advanced Peripheral Bus	Low-power AMBA bus protocol
AMBA	Advanced Microcontroller Bus Architecture	ARM standard bus protocols
CDC	Clock Domain Crossing	Synchronization between clock domains

5.1.2 Signal Acronyms

Acronym	Full Name	Description
CLK	Clock	Timing reference signal
RST	Reset	System initialization signal
OE	Output Enable	Tri-state buffer control
IRQ	Interrupt Request	Hardware interrupt signal

5.1.3 GPIO-Specific Terms

Term	Description
GPIO	General Purpose Input/Output - Configurable digital I/O pins
Pin	Individual GPIO signal (input or output)
Port	Group of 32 GPIO pins managed together

Term	Description
Direction	Input (0) or Output (1) configuration per pin
Polarity	Active-high or active-low signal interpretation

5.1.4 Interrupt Terms

Term	Description
Edge-triggered	Interrupt on signal transition
Level-sensitive	Interrupt while signal is at specified level
Rising edge	Low-to-high transition
Falling edge	High-to-low transition
Both edges	Either transition direction

5.1.5 Register Terms

Term	Description
RW	Read-Write register
RO	Read-Only register
W1C	Write-1-to-Clear register
HWIF	Hardware Interface (PeakRDL generated)

5.2 Navigation

Next: 05_references.md - Reference documents

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

6 APB GPIO - References

6.1 References

6.1.1 Internal Documentation

6.1.1.1 RTL Source Files

Worth having open in an editor while you read the rest of this MAS:

- `rtl/gpio/apb4_gpio.sv` - Top-level module (APB port, CDC generate)
- `rtl/gpio/gpio_config_regs.sv` - Register wrapper (atomic-op pulses, irq)
- `rtl/gpio/gpio_core.sv` - Pin I/O, synchronizers, interrupt detection
- `rtl/gpio/gpio_regs.sv` / `gpio_regs_pkg.sv` - PeakRDL-generated register file
- `rdl/gpio/gpio_regs.rdl` - Register description source

6.1.1.2 Related Specifications

- APB Protocol Specification (AMBA 3)
- RLB Integration Guide

6.1.2 External References

6.1.2.1 ARM AMBA Specifications

- **AMBA 3 APB Protocol Specification**
  - ARM IHI 0024E
  - Defines APB interface timing and protocol

6.1.2.2 Industry Standards

- **GPIO Best Practices**
  - Synchronization for external inputs
  - Interrupt handling patterns
  - Tri-state buffer management

6.1.3 Design References

6.1.3.1 Clock Domain Crossing

- Dual flip-flop synchronizer methodology
- Skid buffer for data path CDC
- Reset synchronization techniques

6.1.3.2 Interrupt Handling

- Edge detection circuits
- Interrupt aggregation patterns
- Software interrupt acknowledge flows

6.2 Navigation

Next: Chapter 2: Block Descriptions

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 APB GPIO - Block Descriptions Overview

7.1 Overview

The design splits into four blocks plus an optional CDC wrapper. Small enough to hold in your head, which is how a GPIO should be.

7.1.1 Figure 2.1: APB GPIO Module Hierarchy

```
apb4_gpio                       top level
├── apb4_slave                  APB completer (CDC_ENABLE=0)
│   or apb4_slave_cdc           APB completer + crossing (CDC_ENABLE=1)
├── peakrdl_to_cmdrsp           cmd/rsp to register-block adapter
├── gpio_config_regs            write strobes, W1C merge, irq
│   ├── gpio_regs               PeakRDL register block
│   └── gpio_core               sync, output latch, edge/level
└── glitch_free_n_dff_arn       irq sync to pclk (CDC_ENABLE=1)
```

7.1.2 Block Summary

Table 2.1: Block Summary

Block	File	Description
APB GPIO Top	apb4_gpio.sv	Top-level module with all GPIO functionality
Register File	gpio_regs.sv (+ gpio_regs_pkg.sv)	PeakRDL-generated control/status registers
Register Wrapper	gpio_config_regs .sv	Connects registers to the core; atomic-op pulses, irq
GPIO Core	gpio_core.sv	Pin I/O, synchronizers, interrupt detection

7.1.3 In This Chapter

7.1.3.1 1. APB Interface

Handles APB protocol conversion and register access.

See: 01_apb_interface.md

7.1.3.2 2. Register File

PeakRDL-generated registers for configuration and status.

See: 02_register_file.md

7.1.3.3 3. GPIO Core

Main GPIO functionality including I/O control and interrupts.

See: 03_gpio_core.md

7.1.3.4 4. Interrupt Controller

Edge detection, level sensing, and interrupt aggregation.

See: 04_interrupt_controller.md

7.1.3.5 5. CDC Logic

Optional clock domain crossing for asynchronous GPIO clock.

See: 05_cdc_logic.md

7.2 Navigation

Next: 01_apb_interface.md - APB Interface details

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

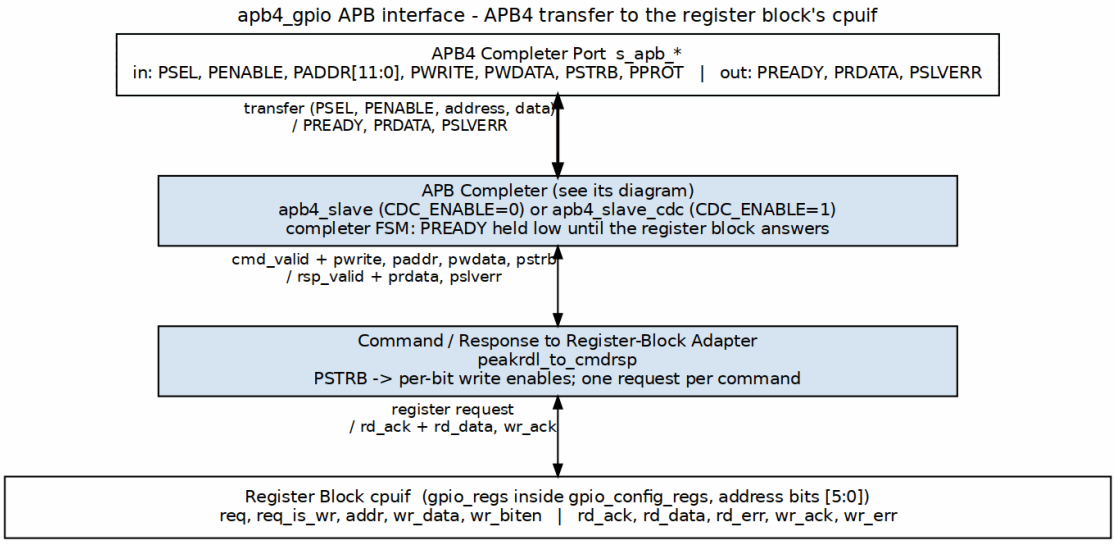
---

8 APB GPIO - APB Interface Block

8.1 Overview

The APB interface is the bridge between the system APB bus and the GPIO register file — a small FSM that turns APB's two-phase transfers into the command/response handshake the register block speaks.

8.1.1 Block Diagram



APB Interface Block

8.2 Ports

8.2.1 APB Slave Interface

Signal	Width	Direction	Description
s_apb_PSEL	1	Input	Slave select
s_apb_PENABLE	1	Input	Enable phase
s_apb_PWRITE	1	Input	Write operation
s_apb_PADDR	12	Input	Address bus
s_apb_PWDATA	32	Input	Write data
s_apb_PSTRB	4	Input	Byte strobes

Signal	Width	Direction	Description
s_apb_PPROT	3	Input	Protection attributes (accepted, unused)
s_apb_PRDATA	32	Output	Read data
s_apb_PREADY	1	Output	Ready response
s_apb_PSLVERR	1	Output	Error response

8.3 Functional Description

8.3.1 Read Transaction

1. Master asserts `psel` and `paddr`
2. Master asserts `penable` on next cycle
3. Slave returns `prdata` with `pready`

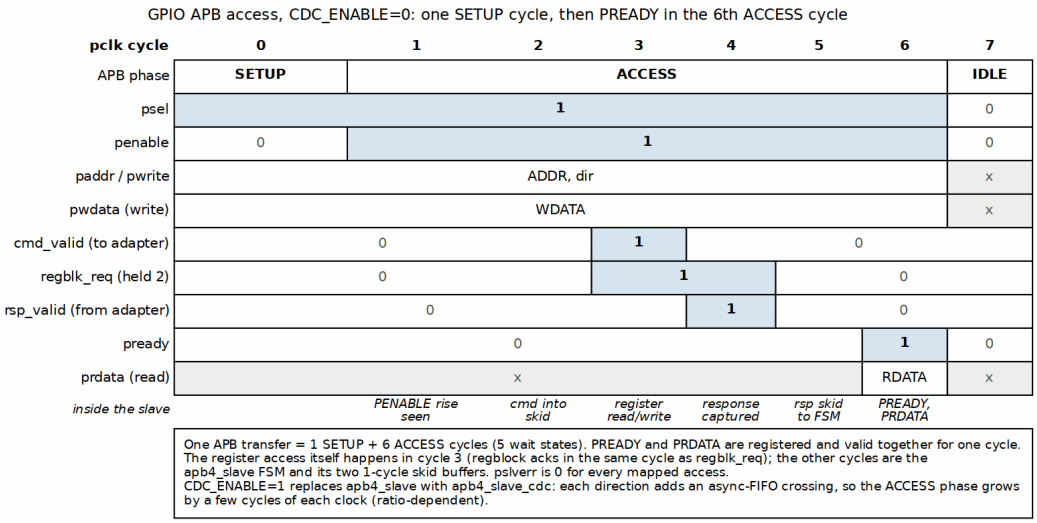
8.3.2 Write Transaction

1. Master asserts `psel`, `paddr`, `pwdata`, `pwrite`
2. Master asserts `penable` on next cycle
3. Slave samples data with `pready`

8.4 Timing

Figure 2.2 shows a basic APB access, start to finish.

8.4.1 Figure 2.2: APB Access Timing



APB Access Timing

One SETUP cycle, then six ACCESS cycles: the register itself is read or written in cycle 3, and PREADY rises with PRDATA in the sixth ACCESS cycle.

8.5 Design Notes

- PREADY-gated: the bridge FSM adds five wait states per access with CDC_ENABLE=0, more with the CDC (no stalls originate in the register block itself)
- No error responses (pslverr always 0)
- 32-bit aligned access only

8.6 Navigation

Next: 02_register_file.md - Register File

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

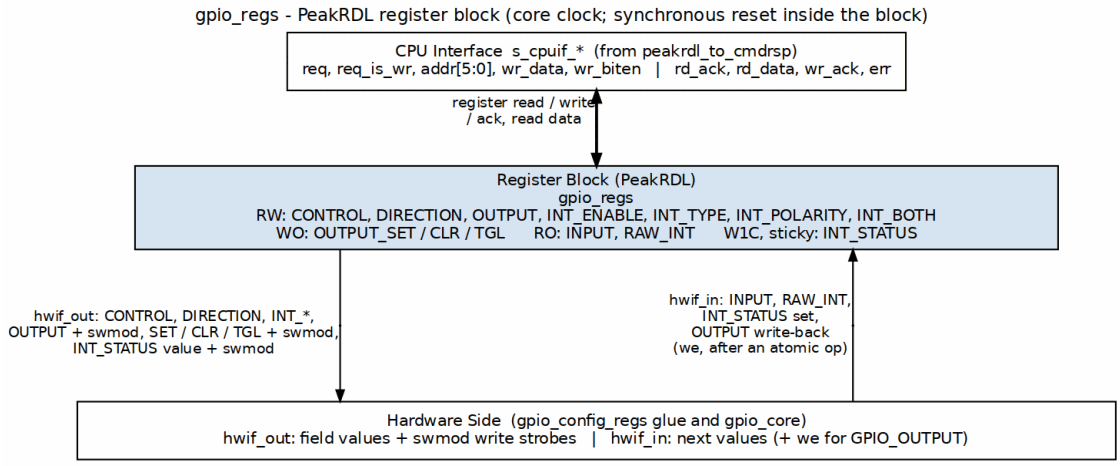
---

9 APB GPIO - Register File Block

9.1 Overview

The register file is generated by PeakRDL from `../../rdl/gpio/gpio_regs.rdl` and is the hardware/software contract for the whole block. If a bit isn't in the RDL, it doesn't exist.

9.1.1 Block Diagram



Register File Block

9.2 Ports

9.2.1 Hardware-to-Software (hw2reg)

Signals the RTL drives into the register file:

Signal	Width	Description
GPIO_INPUT.input_data	32	Current GPIO input values
GPIO_OUTPUT.output_data	32	Live output latch write-back (we-gated, the cycle after an atomic op)
GPIO_INT_STATUS.int_status	32	Interrupt status set requests (sticky; includes the deferred W1C-coincident set)
GPIO_RAW_INT.raw_status	32	Live raw event-detector status

9.2.2 Software-to-Hardware (reg2hw)

Signals the register file drives out to the RTL:

Signal	Width	Description
GPIO_CONTROL.gpio_enable	1	GPIO output enable (gates gpio_oe only; resets 1)
GPIO_CONTROL.int_enable	1	Global interrupt enable (gates irq; resets 0)
GPIO_DIRECTION.direction	32	Pin direction (0=in, 1=out)
GPIO_OUTPUT.output_data	32	Output values (+ swmod write strobe)
GPIO_INT_ENABLE.int_enable	32	Interrupt enable per pin
GPIO_INT_TYPE.int_type	32	Interrupt type (0=edge, 1=level)
GPIO_INT_POLARITY.int_polarity	32	Polarity (0=fall/low, 1=rise/high)
GPIO_INT_BOTH.int_both	32	Both edges enable
GPIO_OUTPUT_SET.set_bits	32	Atomic set mask (+ swmod write strobe)
GPIO_OUTPUT_CLR.clear_bits	32	Atomic clear mask (+ swmod write strobe)
GPIO_OUTPUT_TGL.toggle_bits	32	Atomic toggle mask (+ swmod write strobe)
GPIO_INT_STATUS.int_status	32	Sticky status value (+ swmod W1C write strobe)

9.3 Functional Description

9.3.1 Byte Enable Support

All registers support byte-granular writes via pstrb: - `pstrb[0]` enables bits [7:0] - `pstrb[1]` enables bits [15:8] - `pstrb[2]` enables bits [23:16] - `pstrb[3]` enables bits [31:24]

9.3.2 Access Types

Type	Read Behavior	Write Behavior
RW	Returns current value	Updates register
RO	Returns hardware value	No effect
W1C	Returns current value	Clears bits where 1 written

9.4 Design Notes

- Generated by PeakRDL regblock
- Clocked by the core clock (w_core_clk = gpio_clk when CDC_ENABLE=1, pclk otherwise – see the CDC chapter)
- Registers reset to 0 except GPIO_CONTROL.GPIO_ENABLE (resets to 1) and GPIO_INT_POLARITY (resets to 0xFFFFFFFF); reset is synchronous inside the generated block
- `swmod` on GPIO_OUTPUT, GPIO_OUTPUT_SET/CLR/TGL and GPIO_INT_STATUS is a level held for the whole command-bridge transaction and leads the field storage by one cycle; `gpio_config_regs` edge-detects and re-aligns it before use, so every write is one operation (see Chapter 5)

9.5 Navigation

Next: 03_gpio_core.md - GPIO Core

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

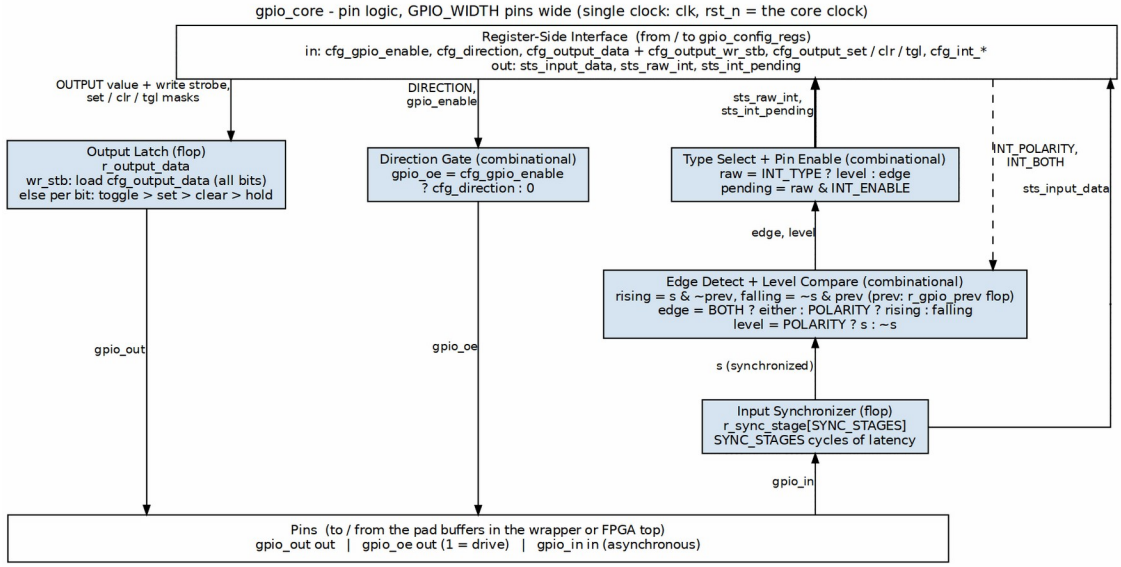
---

10 APB GPIO - GPIO Core Block

10.1 Overview

The core is where the pins live: input synchronization, output driving, and direction control for all 32 GPIOs.

10.1.1 Block Diagram



GPIO Core Block

10.2 Ports

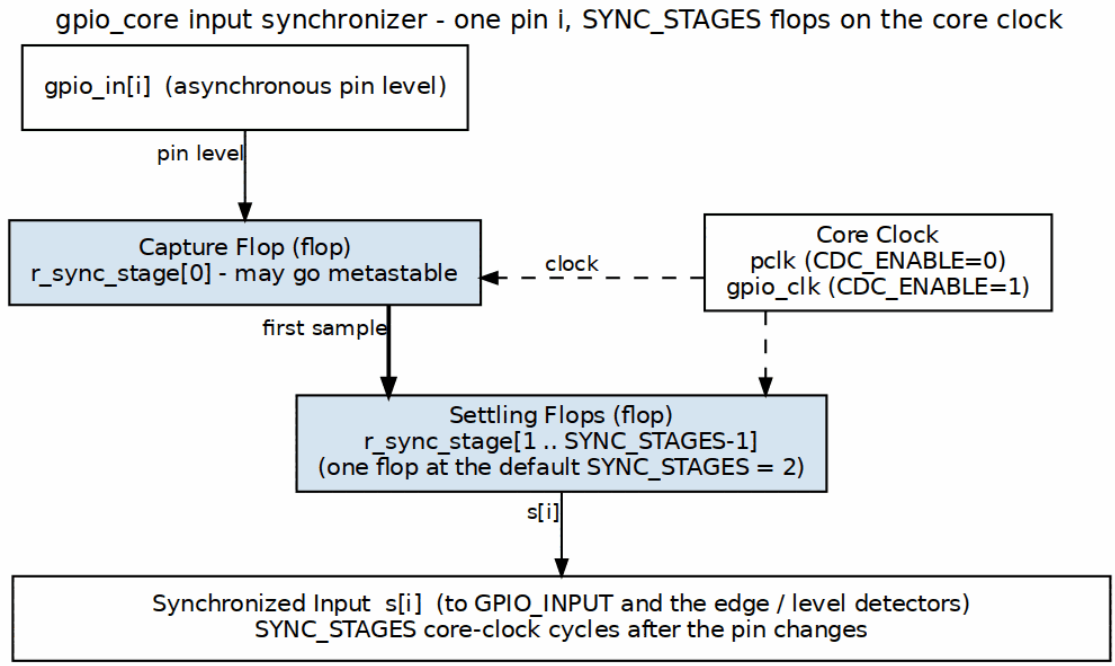
Signal	Width	Direction	Description
gpio_out	32	Output	Output data values
gpio_oe	32	Output	Output enables (active high)
gpio_in	32	Input	Input data values

10.3 Functional Description

10.3.1 Input Path

10.3.1.1 Synchronization

External inputs pass through a dual flip-flop synchronizer:



Input Synchronizer

- Prevents metastability from asynchronous inputs
- Configurable depth via `SYNC_STAGES` parameter
- Adds SYNC_STAGES clock cycles of latency

10.3.1.2 Input Register

Synchronized inputs land in the `GPIO_INPUT` register, which is what software reads.

10.3.2 Output Path

10.3.2.1 Output Register

Software writes to `GPIO_OUTPUT` to set output values. The write reaches the core as a one-cycle strobe (`cfg_output_wr_stb`, built in `gpio_config_regs` from the register's write strobe) alongside the data, so it is a write whether or not the value changed. A direct write takes priority over an atomic operation in the same cycle (direct > toggle > set > clear):

```
if (cfg_output_wr_stb)        r_output_data <= cfg_output_data;
else                          // toggle, then set, then clear
```

`r_output_data` drives the pins and is written back into `GPIO_OUTPUT` after each atomic operation, so the register reads the live value.

10.3.2.2 Output Enable

The direction register controls the tri-state buffers: - `direction[i] = 0`: Pin is input (high-Z output) - `direction[i] = 1`: Pin is output (driven)

10.3.3 Direction Control

10.3.3.1 Per-Pin Configuration

Each pin is configured independently:

```
// gpio_oe = cfg_gpio_enable ? cfg_direction : '0;   (gpio_core.sv)
if (gpio_enable && direction[i]) begin
    // Output mode
    gpio_oe[i] = 1'b1;
    gpio_out[i] = output_reg[i];
end else begin
    // Input mode, or GPIO_CONTROL.ENABLE == 0 (all pins high-Z)
    gpio_oe[i] = 1'b0;
    // gpio_out[i] = don't care
end
```

GPIO_CONTROL.ENABLE gates ONLY the output enables: when it is 0 every pin is high-Z, but output data, input synchronization, and interrupt detection/status all keep running.

10.3.3.2 Read-Back Behavior

Reading `GPIO_INPUT` returns: - For input pins: External signal value (synchronized) - For output pins: External signal value (may differ from output_reg if open-drain)

10.4 Design Notes

- All 32 pins processed in parallel
- Output updates one core-clock after the GPIO_OUTPUT field updates (the write strobe is aligned to the field, and `r_output_data` registers it)
- The core exports `sts_raw_int` and `sts_int_pending` only; the aggregate `irq` is built in `gpio_config_regs` from the sticky status register and the live level pins. The core has no irq output and no global-enable input
- Input synchronization always active

10.5 Navigation

Next: 04_interrupt_controller.md - Interrupt Controller

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

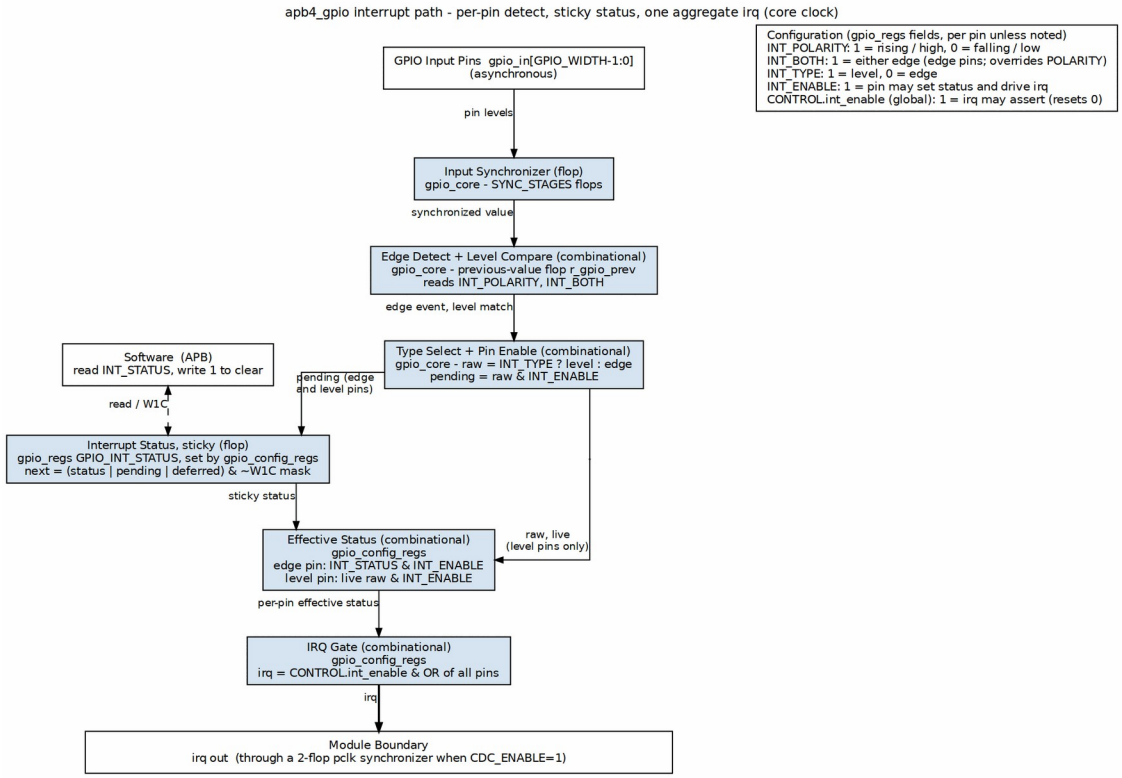
---

11 APB GPIO - Interrupt Controller Block

11.1 Overview

Per-pin interrupt generation with edge or level triggering and one aggregate `irq`. The logic is simple; the enable semantics are where people trip. There are two levels of enable and they don't mean the same thing — read that section twice.

11.1.1 Figure 2.5: Interrupt Controller Block Diagram



Interrupt Controller Block

11.2 Functional Description

11.2.1 Interrupt Modes

11.2.1.1 Edge-Triggered Mode

```
GPIO_INT_TYPE[i] = 0
```

Table 2.5: Edge-Triggered Modes

GPIO_INT_POLARITY	GPIO_INT_BOTH	Trigger Condition
0	0	Falling edge only
1	0	Rising edge only
X	1	Both edges

11.2.1.2 Level-Sensitive Mode

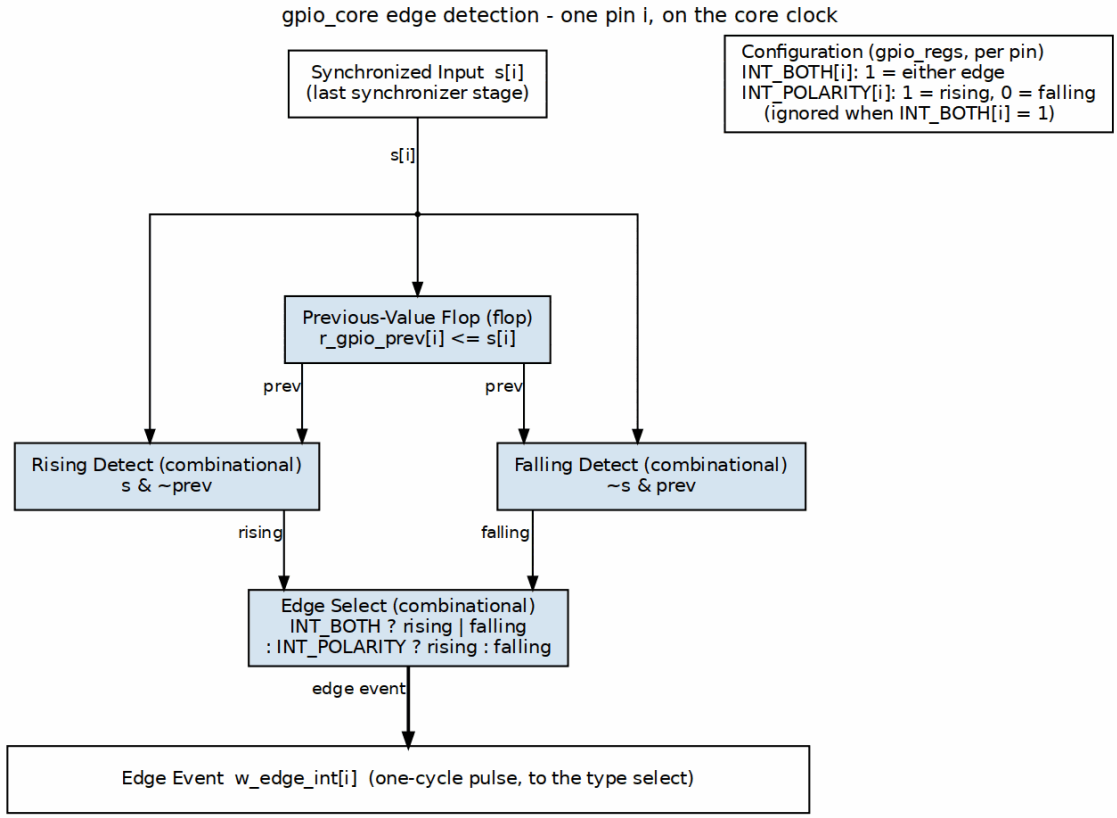
```
GPIO_INT_TYPE[i] = 1
```

Table 2.6: Level-Sensitive Modes

GPIO_INT_POLARITY	Trigger Condition
0	Active low (interrupt while pin = 0)
1	Active high (interrupt while pin = 1)

11.2.2 Edge Detection Logic

11.2.2.1 Figure 2.6: Edge Detection Logic



Edge Detection Logic

11.2.3 Interrupt Status

- Each bit in `GPIO_INT_STATUS` corresponds to one pin
- Set when the interrupt condition is detected on a pin whose `GPIO_INT_ENABLE` bit is 1 - events on disabled pins set nothing (`sts_int_pending = raw & enable` in `gpio_core.sv`)
- Cleared by writing 1 to the bit (W1C); bits latch in both edge and level modes until cleared
- A W1C write and a hardware event in the same cycle merge per bit (`next = (value | hw_set) & ~w1c_mask`, `gpio_config_regs.sv`): the event survives on every bit the write does not clear

11.2.4 Interrupt Enable

Two levels of enable exist:

- `GPIO_INT_ENABLE[i] = 1` enables pin i. It is sampled at EVENT time: it gates whether a status bit sets, and (for level pins) whether the pin drives `irq`. It is also re-applied at `irq`: clearing it after an edge event has latched masks that pending interrupt (the status bit stays set until W1C), as described under Aggregate IRQ Output below.
- `GPIO_CONTROL[1]` (global INT_ENABLE) gates the final `irq` output and is the only control that masks everything. It resets to 0.

11.2.5 Aggregate IRQ Output

```
per-pin effective status:
  level pin (TYPE=1): raw_detector & GPIO_INT_ENABLE   (live, bypasses
STATUS)
  edge  pin (TYPE=0): GPIO_INT_STATUS & GPIO_INT_ENABLE (sticky latch,
re-gated)

irq = GPIO_CONTROL[1] && (| effective_status)
```

(`gpio_config_regs.sv`.) One difference from a plain `|(STATUS & ENABLE)` remains: level pins bypass GPIO_INT_STATUS entirely, so W1C on a level pin does not deassert `irq` while the level persists. Edge pins re-apply the current per-pin enable, so clearing GPIO_INT_ENABLE[i] masks an already-latched edge interrupt on pin i (the status bit stays set until W1C and `irq` re-asserts if the enable is set again).

11.2.6 Interrupt Handling Flow

1. Hardware detects condition, sets status bit
2. IRQ asserted to processor
3. Software reads `GPIO_INT_STATUS` to identify source
4. Software handles interrupt
5. Software writes 1 to status bit to clear
6. IRQ deasserts (if no other sources active)

11.3 Design Notes

- Edge detection uses synchronized input
- Level-sensitive interrupts re-trigger if not cleared
- Status bits latch until software clears

11.4 Navigation

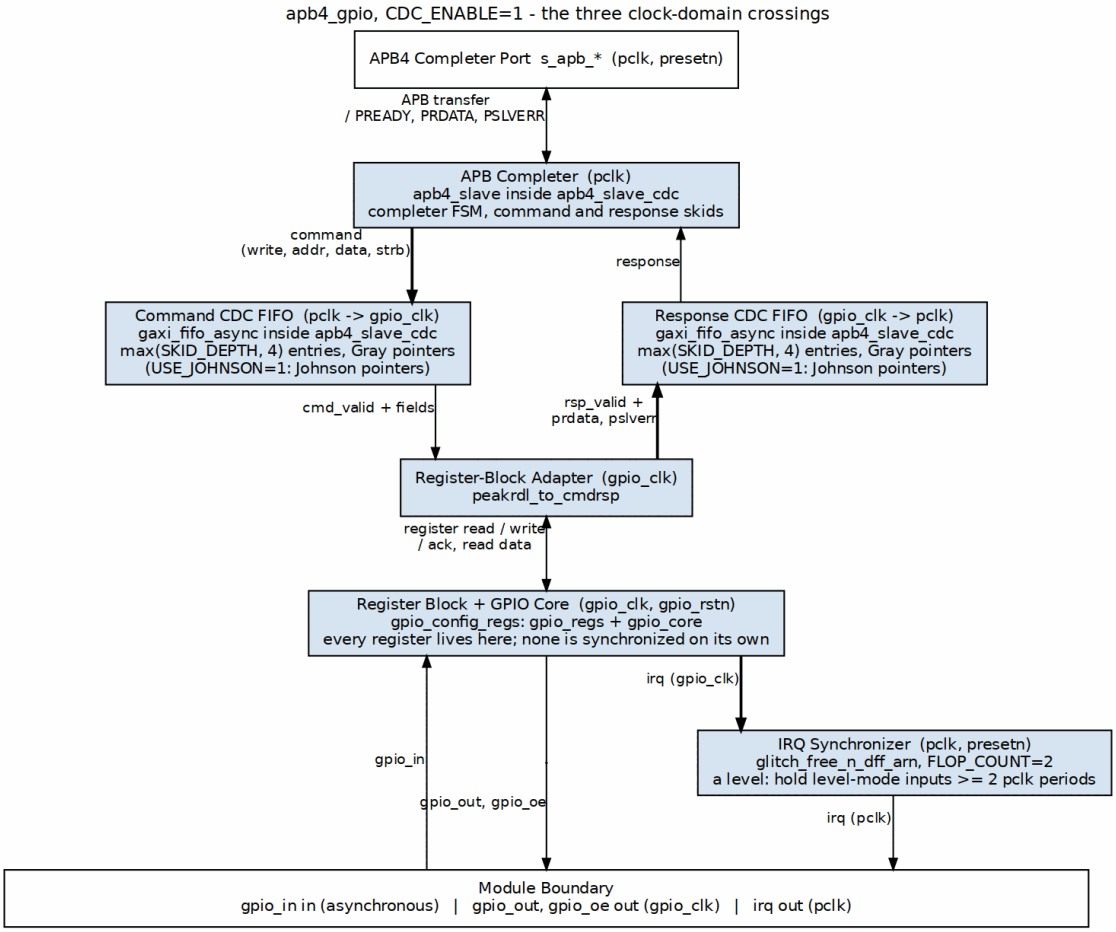
Next: 05_cdc_logic.md - CDC Logic

12 APB GPIO - CDC Logic Block

12.1 Overview

When you need the GPIO core on its own clock, asynchronous to the APB bus, `CDC_ENABLE` moves the whole register file across the domain boundary. Everything about the crossing lives in this one block.

12.1.1 Figure 2.7: CDC Logic Block Diagram



CDC Logic Block

12.2 Parameters

12.2.1 CDC_ENABLE

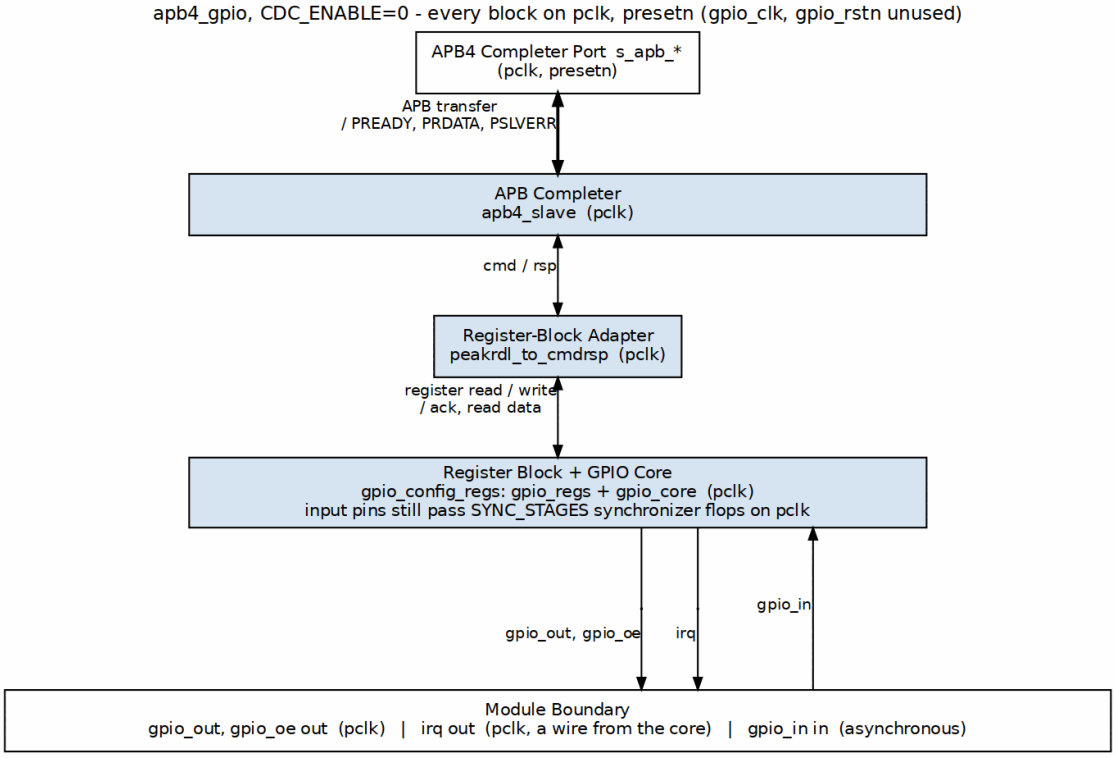
Table 2.7: CDC Enable Parameter

Value	Behavior
0	Single clock domain, all logic on pclk
1	Dual clock domain, GPIO core on gpio_clk

12.3 Functional Description

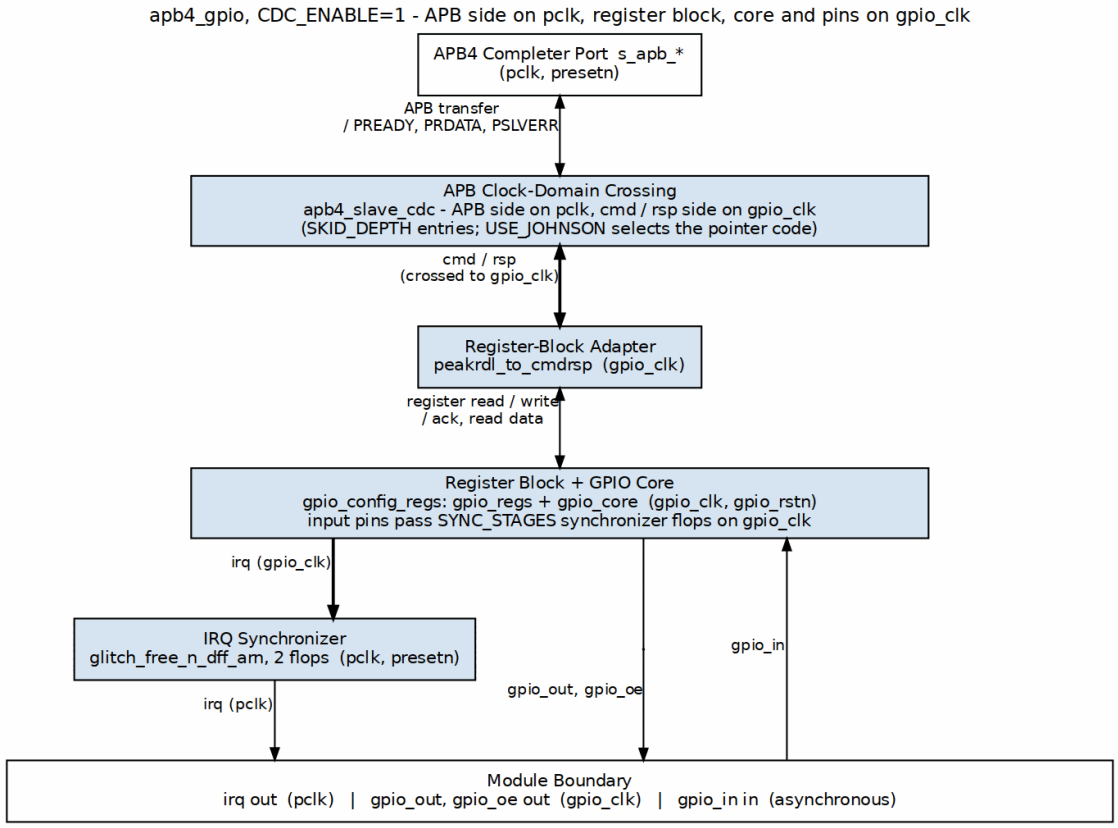
12.3.1 Clock Domains

12.3.1.1 Figure 2.8: Single Clock Domain (CDC Disabled)



CDC Disabled

12.3.1.2 *Figure 2.9: Dual Clock Domain (CDC Enabled)*



CDC Enabled

12.3.2 CDC Implementation

When CDC_ENABLE=1 the ENTIRE register file and GPIO core sit in the gpio_clk domain; no register value is individually synchronized. The only crossing is the APB command/response stream itself, carried through an `apb4_slave_cdc` instance at the bus boundary (`apb4_gpio.sv`, `gen_cdc`). An APB access therefore completes with CDC handshake latency, and once it lands in the gpio_clk domain every register behaves exactly as in the single-clock configuration.

12.3.3 The `irq` Output

`irq` is generated in the core clock domain (`gpio_config_regs.sv`), which is gpio_clk when CDC_ENABLE=1. `apb4_gpio.sv` (`gen_irq_sync`) then passes it through a 2-flop synchronizer (`glitch_free_n_dff_arn`, FLOP_COUNT=2) clocked by pclk, so the output pin is safe to consume in the pclk domain with no external synchronizer. It is a level, so a plain synchronizer is the right primitive; there is no handshake and no pulse stretching. The cost is two pclk cycles of latency. When CDC_ENABLE=0 the synchronizer is bypassed (`gen_irq_direct`) and `irq` keeps the zero-latency behaviour of the single-clock configuration.

One contract comes with that choice. For an edge-mode pin the sticky status bit holds `irq` asserted until software clears it, so the synchronizer always sees it. For a level-mode pin `irq` is only as wide as the input level itself, and a level that de-asserts within about two pclk periods (plus the input synchronizer depth) can pass between the synchronizer's samples and never reach the pin. The event is not lost – GPIO_INT_STATUS still latches it and polling sees it – but no interrupt fires. Hold level-mode inputs for at least two pclk periods when CDC_ENABLE=1, or use edge mode for short pulses. With CDC_ENABLE=0 every core-clock assertion is visible, so the constraint only exists in the asynchronous configuration.

12.3.4 Coherency

- No guaranteed atomicity across clock domains
- Software must handle potential inconsistencies
- Interrupt status always reflects gpio_clk domain

12.3.5 Reset Synchronization

Both resets must be asserted at power-on: 1. Assert both `presetn` and `gpio_rstn` 2. Release `gpio_rstn` first 3. Release `presetn` after gpio_clk domain stable

12.4 Timing

12.4.1 Latency

Table 2.8: CDC Latency

Path	Latency
APB write to gpio_clk-domain register	APB access + apb4_slave_cdc handshake (a few cycles of each clock)
APB read of gpio_clk-domain state	Same crossing, in both directions
Interrupt detection to IRQ	gpio_clk-domain detection (Chapter 3.3) + 2 pclk cycles through the irq synchronizer

12.5 Navigation

Back to: 00_overview.md - Block Descriptions Overview

Next Chapter: Chapter 3: Interfaces

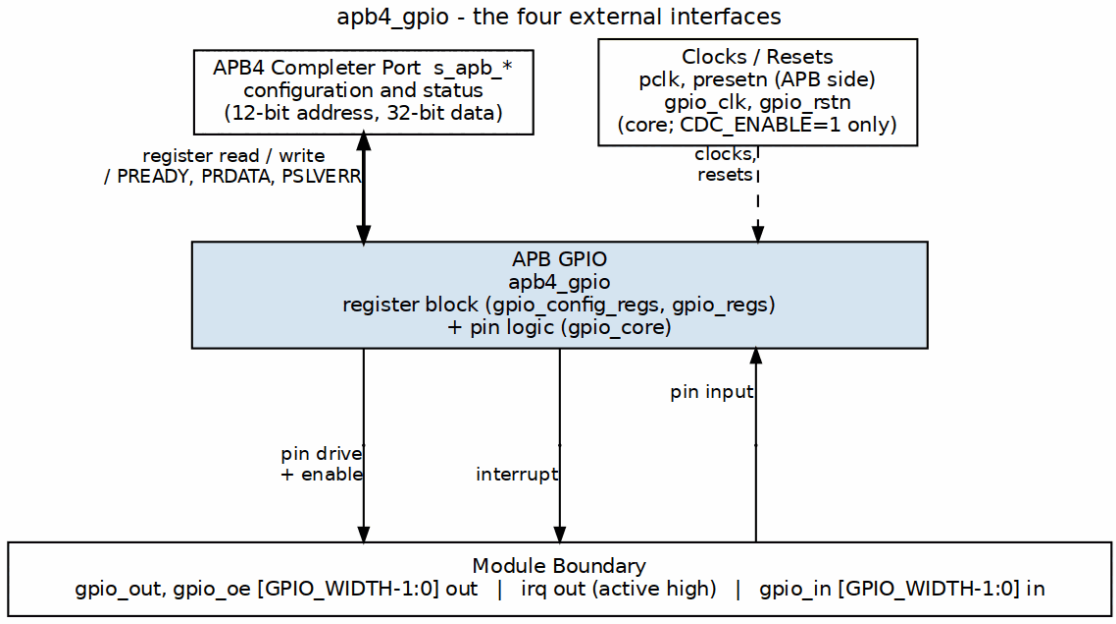
13 APB GPIO - Interfaces Overview

13.1 Overview

Four external interfaces: the bus, the pins, the interrupt, and the clock/reset pair.

Interface	Type	Description
APB Slave	Bus	Configuration and status access
GPIO Pins	I/O	32 general-purpose I/O pins
Interrupt	Signal	Aggregate interrupt output
Clocks/Reset	System	Clock and reset inputs

13.1.1 Interface Summary Diagram



GPIO Interfaces

13.1.2 In This Chapter

13.1.2.1 APB Slave Interface

Complete APB protocol interface for register access.

See: 01_apb4_slave.md

13.1.2.2 GPIO Pin Interface

External GPIO pin connections with tri-state control.

See: 02_gpio_pins.md

13.1.2.3 Interrupt Interface

Interrupt request output signal.

See: 03_interrupt.md

13.1.2.4 System Interface

Clock and reset signal requirements.

See: 04_system.md

13.2 Navigation

Next: 01_apb4_slave.md - APB Slave Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 APB GPIO - APB Slave Interface

14.1 Ports

14.1.1 APB Slave Signals

Signal	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)
s_apb_PSEL	1	Input	Peripheral select
s_apb_PENABLE	1	Input	Enable phase
s_apb_PWRITE	1	Input	Write transaction
s_apb_PADDR	12	Input	Address bus
s_apb_PWDATA	32	Input	Write data
s_apb_PSTRB	4	Input	Byte strobes
s_apb_PRDATA	32	Output	Read data
s_apb_PREADY	1	Output	Ready response
s_apb_PSLVERR	1	Output	Slave error

14.2 Functional Description

14.2.1 Protocol Compliance

APB3/APB4 feature support:

Feature	Support
PSEL	Yes
PENABLE	Yes
PWRITE	Yes

Feature	Support
PADDR	12-bit
PWDATA	32-bit
PRDATA	32-bit
PREADY	Yes (inserts wait states – see Access Timing below)
PSLVERR	Yes (always 0)
PSTRB	Yes
PPROT	Present on the port (`s_apb_PPROT[2:0]`), accepted and ignored - no protection checking is performed

14.2.2 Address Decoding

Offset	Name	Access
0x000	GPIO_CONTROL	RW
0x004	GPIO_DIRECTION	RW
0x008	GPIO_OUTPUT	RW
0x00C	GPIO_INPUT	RO
0x010	GPIO_INT_ENABLE	RW
0x014	GPIO_INT_TYPE	RW
0x018	GPIO_INT_POLARITY	RW
0x01C	GPIO_INT_BOTH	RW
0x020	GPIO_INT_STATUS	W1C
0x024	GPIO_RAW_INT	RO
0x028	GPIO_OUTPUT_SET	WO
0x02C	GPIO_OUTPUT_CLR	WO
0x030	GPIO_OUTPUT_TGL	WO

Only PADDR[5:0] reaches the register block, so this map aliases every 64 bytes across the 12-bit APB window; 0x034-0x03F read as zero. No address ever raises PSLVERR.

14.2.3 Byte Strobes

Byte-granular writes supported: - pstrb[3:0] corresponds to pwdata[31:0] - Unselected bytes retain previous values

14.2.4 Error Handling

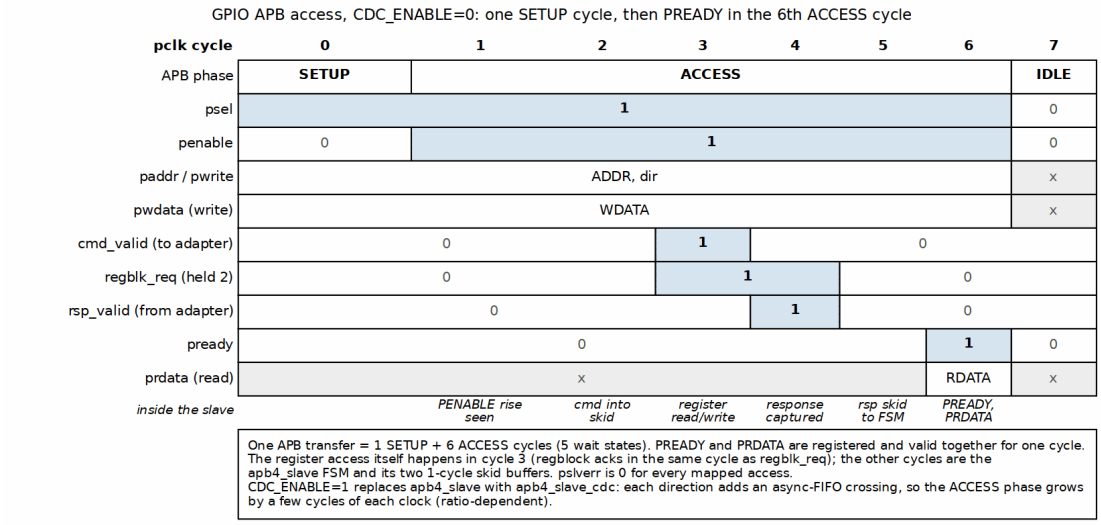
- No address decode errors (all addresses valid)
- No timeout errors
- pslverr always 0

14.3 Timing

14.3.1 Access Timing

PREADY inserts wait states: the apb4_slave bridge is a small FSM that issues the command and returns the response over a registered cmd/rsp handshake, so an access takes five pclk wait states beyond the 2-cycle APB minimum (more when CDC_ENABLE=1 crosses to gpio_clk). The register block itself never stalls (`cpuif_req_stall_* = 0`).

14.3.2 Figure 3.1: APB Access Timing (CDC_ENABLE=0)



APB Access Timing

The same access as Figure 2.2: PREADY and PRDATA arrive together in the sixth ACCESS cycle, and PWDATA is held from SETUP until PREADY.

14.4 Navigation

Next: 02_gpio_pins.md - GPIO Pin Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 APB GPIO - GPIO Pin Interface

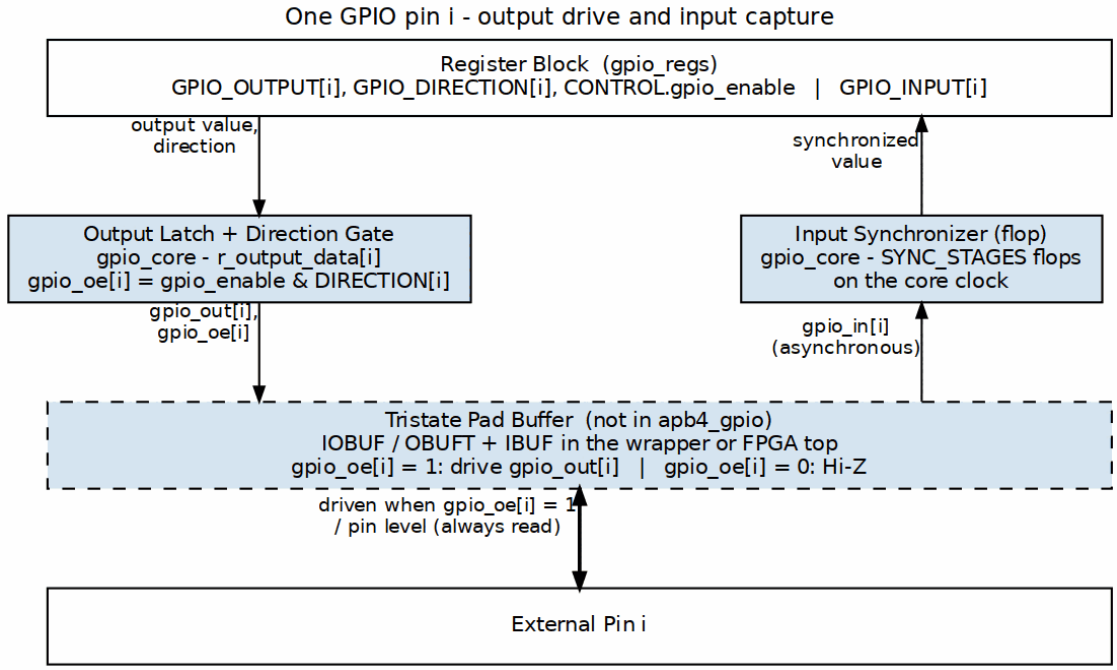
15.1 Ports

15.1.1 GPIO Signals

Signal	Width	Direction	Description
gpio_out	32	Output	Output data values
gpio_oe	32	Output	Output enables
gpio_in	32	Input	Input data values

15.2 Functional Description

15.2.1 Output Mode (direction[i] = 1)



One GPIO Pin

- gpio_oe[i] = 1 (output enabled)
- gpio_out[i] = GPIO_OUTPUT register value
- Pin driven to output value

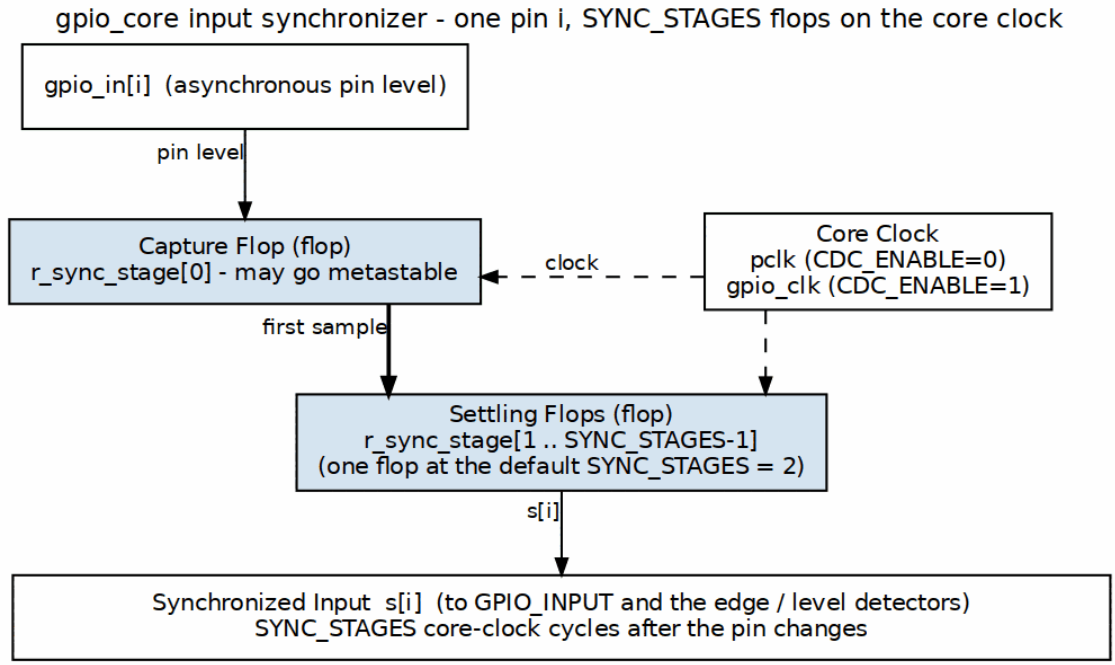
15.2.2 Input Mode (direction[i] = 0)

The same path as the figure above with `gpio_oe[i] = 0`: the pad buffer is Hi-Z and only the input leg, through the synchronizer, is live.

- gpio_oe[i] = 0 (tri-state)
- gpio_out[i] = don't care
- External value captured via synchronizer

15.2.3 Input Synchronization

All inputs pass through a dual flip-flop synchronizer:



Input Synchronizer

- SYNC_STAGES parameter controls depth (default: 2)
- Prevents metastability from asynchronous inputs
- All 32 inputs synchronized in parallel

Input changes show up in the GPIO_INPUT register after: - SYNC_STAGES cycles of gpio_clk (or pclk if CDC_ENABLE=0) - Plus APB read latency

15.2.4 Electrical Considerations

15.2.4.1 Output Characteristics

Parameter	Description
Drive	Standard CMOS output
Slew	Defined by I/O cell
Protection	ESD per I/O cell design

15.2.4.2 Input Characteristics

Parameter	Description
Levels	CMOS compatible
Hysteresis	Optional per I/O cell
Pull-up/down	External to GPIO module

15.3 Timing

15.3.1 Output Path

- gpio_out updates on clock edge
- gpio_oe updates on same clock edge
- No glitch-free guarantee during direction change

15.3.2 Input Path

- Setup/hold relative to synchronizer clock
- Metastability resolved by synchronizer

15.4 Navigation

Next: 03_interrupt.md - Interrupt Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 APB GPIO - Interrupt Interface

16.1 Ports

Signal	Width	Direction	Description
irq	1	Output	Interrupt request (active high)

16.2 Functional Description

16.2.1 Interrupt Generation

```
per-pin effective status:
  level pin (TYPE=1): raw_detector[i] & GPIO_INT_ENABLE[i]     (live)
  edge  pin (TYPE=0): GPIO_INT_STATUS[i] & GPIO_INT_ENABLE[i]  (sticky
latch)

irq = GPIO_CONTROL[1] && (| effective_status)
```

IRQ requires the global interrupt enable GPIO_CONTROL[1] (resets to 0). Level pins drive `irq` from the live detector output, bypassing GPIO_INT_STATUS; edge pins drive it from the latched status re-gated by the current per-pin enable, so clearing GPIO_INT_ENABLE[i] masks an already-latched edge interrupt on that pin without disturbing its status bit.

16.2.2 Per-Pin Configuration

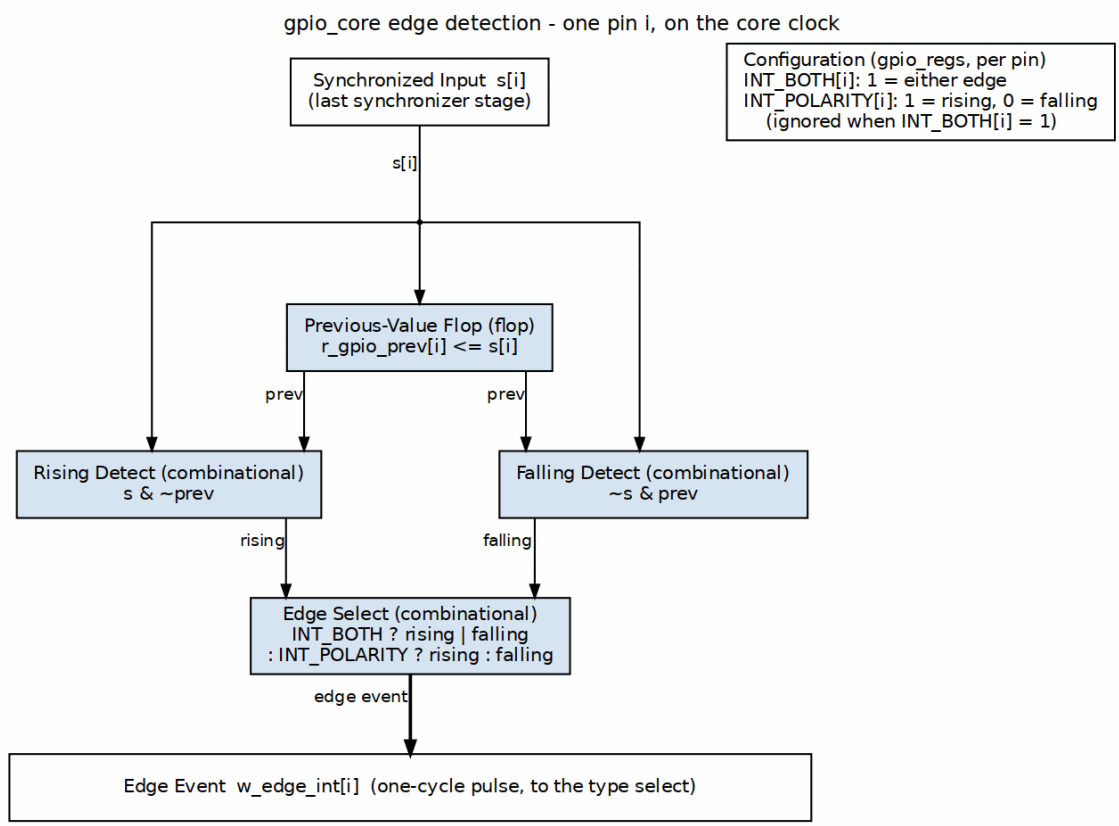
Each GPIO pin can generate interrupts independently:

Register	Function
GPIO_INT_ENABLE	Enable/disable per pin
GPIO_INT_TYPE	Edge (0) or Level (1)
GPIO_INT_POLARITY	Falling/Low (0) or Rising/High (1)
GPIO_INT_BOTH	Both edges (edge mode only)
GPIO_INT_STATUS	Current interrupt status

16.2.3 Interrupt Modes

16.2.3.1 Edge-Triggered

GPIO_INT_TYPE[i] = 0

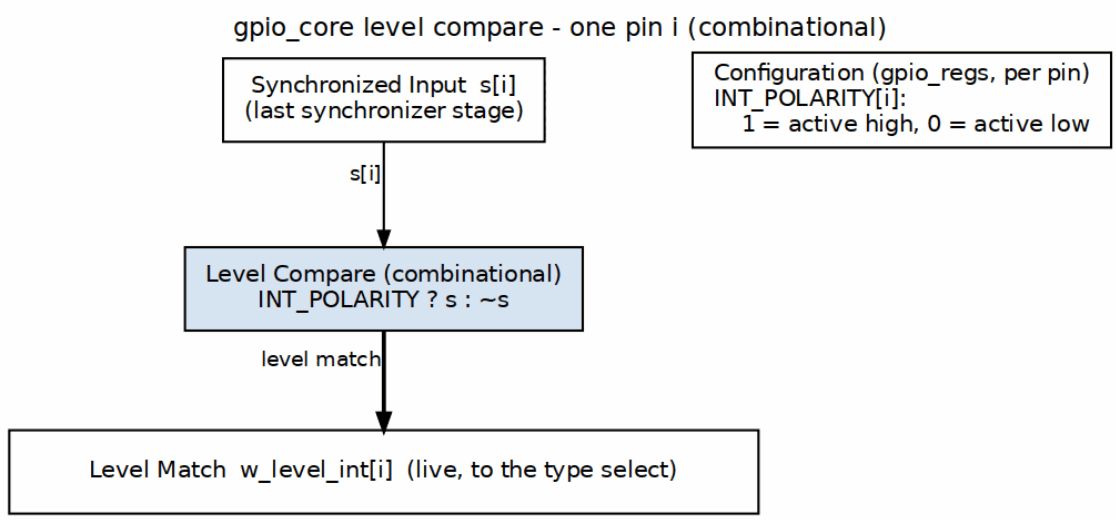


Edge Detection

- Captures transitions on synchronized input
- Status bit latches until cleared by software
- Both-edge mode ignores polarity setting

16.2.3.2 Level-Sensitive

GPIO_INT_TYPE[i] = 1



Level Compare

- Continuously compares input to polarity
- The STATUS bit is sticky: it latches on the (enabled) level event and stays set after the level clears, until W1C - same latching as edge mode
- What follows the input level is `irq` itself, because level pins drive `irq` from the live detector rather than from GPIO_INT_STATUS

16.2.4 Interrupt Handling

16.2.4.1 Software Sequence

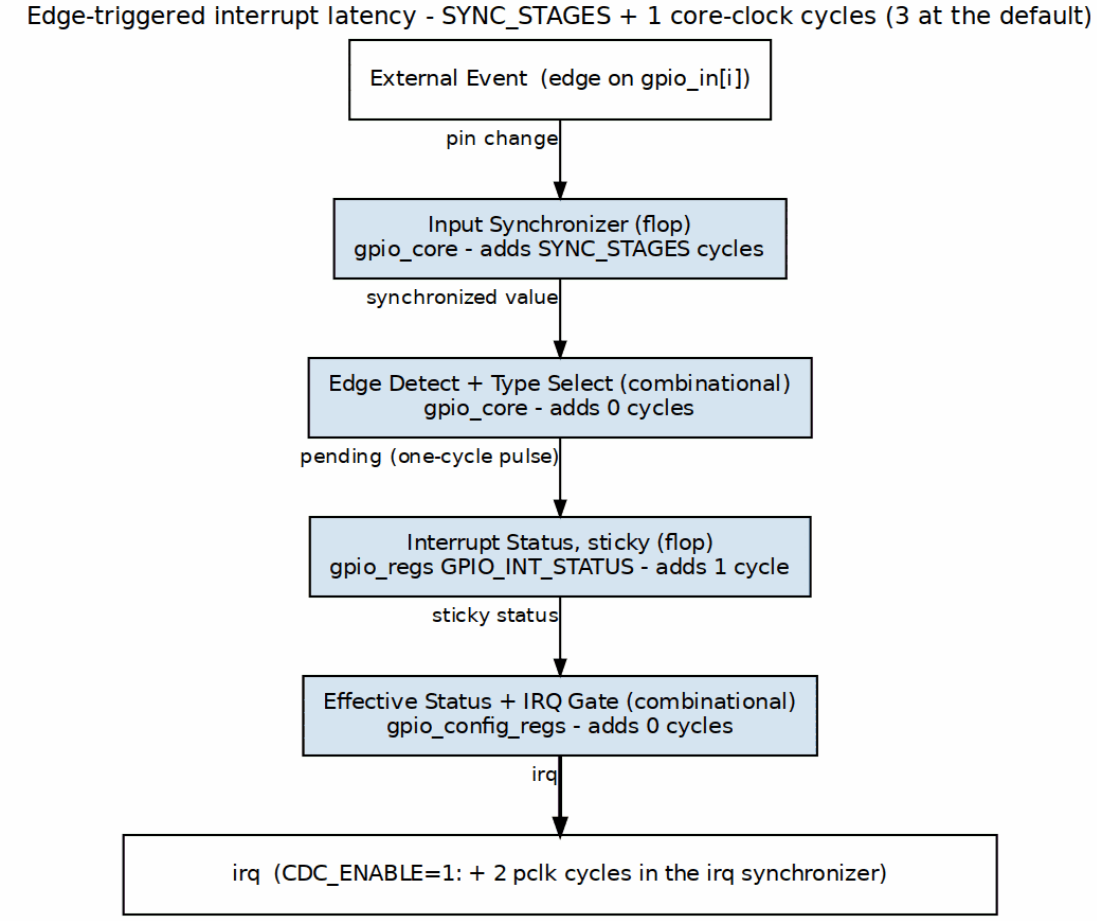
1. IRQ asserts (hardware)
2. CPU vectors to interrupt handler
3. Read GPIO_INT_STATUS to identify sources
4. Handle interrupt condition
5. Write 1 to GPIO_INT_STATUS bits to clear
6. IRQ deasserts if no other sources

16.2.4.2 Clearing Interrupts

Mode	Clear Method
Edge	Write 1 to STATUS bit
Level	Clear source, then write 1 to STATUS

16.3 Timing

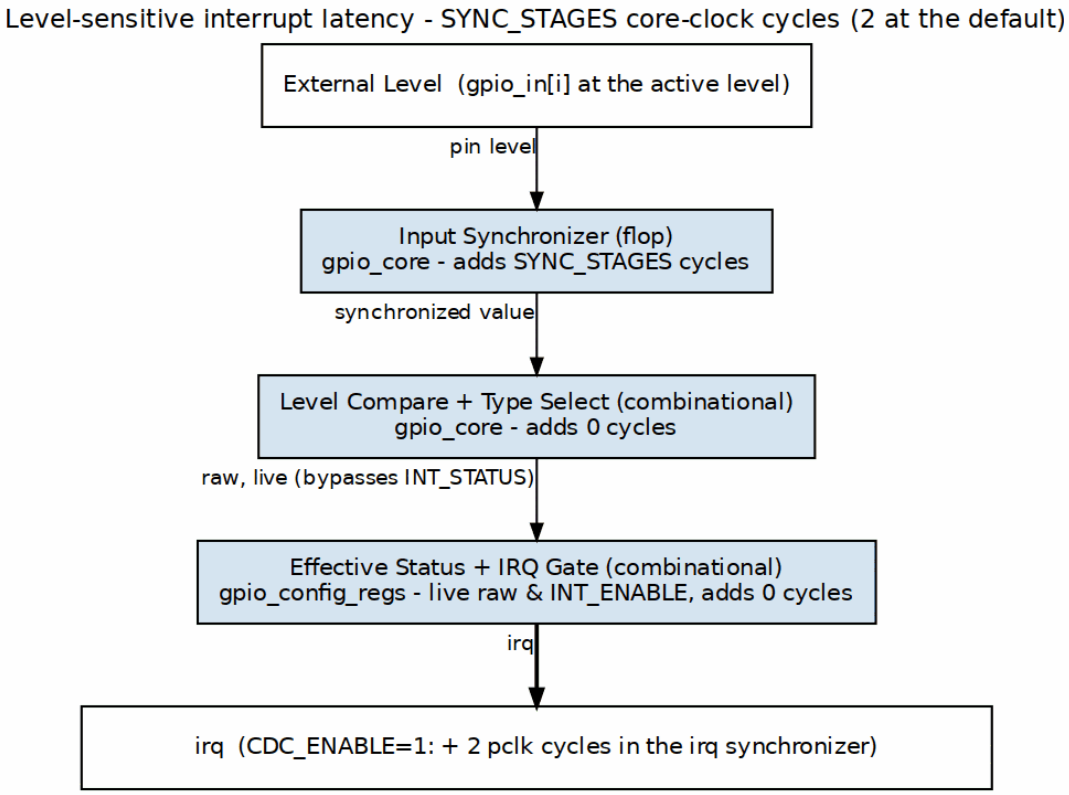
16.3.1 Edge-Triggered Latency



Edge-Triggered Latency

Total: 3 clock cycles typical (SYNC_STAGES=2 + 1 sticky-flop; edge detect adds no cycle)

16.3.2 Level-Sensitive Latency



Level-Sensitive Latency

Total: 2 clock cycles typical (level irq is combinational from the synchronizer output)

Both totals are core-clock cycles. With CDC_ENABLE=1 the core clock is gpio_clk and `irq` then crosses a 2-flop synchronizer into pclk (`apb4_gpio.sv`, `gen_irq_sync`), adding two pclk cycles; with CDC_ENABLE=0 there is no crossing and nothing is added.

16.4 Design Notes

16.4.1 Connection Guidelines

- Connect to interrupt controller input
- Active-high, level-sensitive recommended at controller
- Single IRQ covers all 32 GPIO pins
- Already in the pclk domain when CDC_ENABLE=1 (synchronized inside `apb4_gpio`); no external synchronizer needed

16.5 Navigation

Next: 04_system.md - System Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 APB GPIO - System Interface

17.1 Ports

17.1.1 pclk - APB Clock

Parameter	Value
Purpose	APB interface clock
Frequency	50-200 MHz typical
Domain	APB bus timing

Used for: - APB protocol timing - Register file access - `irq` output (generated on pclk when CDC_ENABLE=0; synchronized into pclk when CDC_ENABLE=1)

17.1.2 gpio_clk - GPIO Clock

Parameter	Value
Purpose	GPIO core clock
Frequency	Application dependent
Usage	Only when CDC_ENABLE=1

Used for: - Input synchronization - Output register updates - Interrupt detection

17.1.3 presetn - APB Reset

Parameter	Value
Polarity	Active low
Type	Asynchronous assert, synchronous deassert (see note)
Scope	APB interface logic

Resets: - APB state machine - Register file - Response logic

17.1.4 gpio_rstn - GPIO Reset

Parameter	Value
Polarity	Active low
Type	Asynchronous assert, synchronous deassert (see note)
Usage	Only when CDC_ENABLE=1

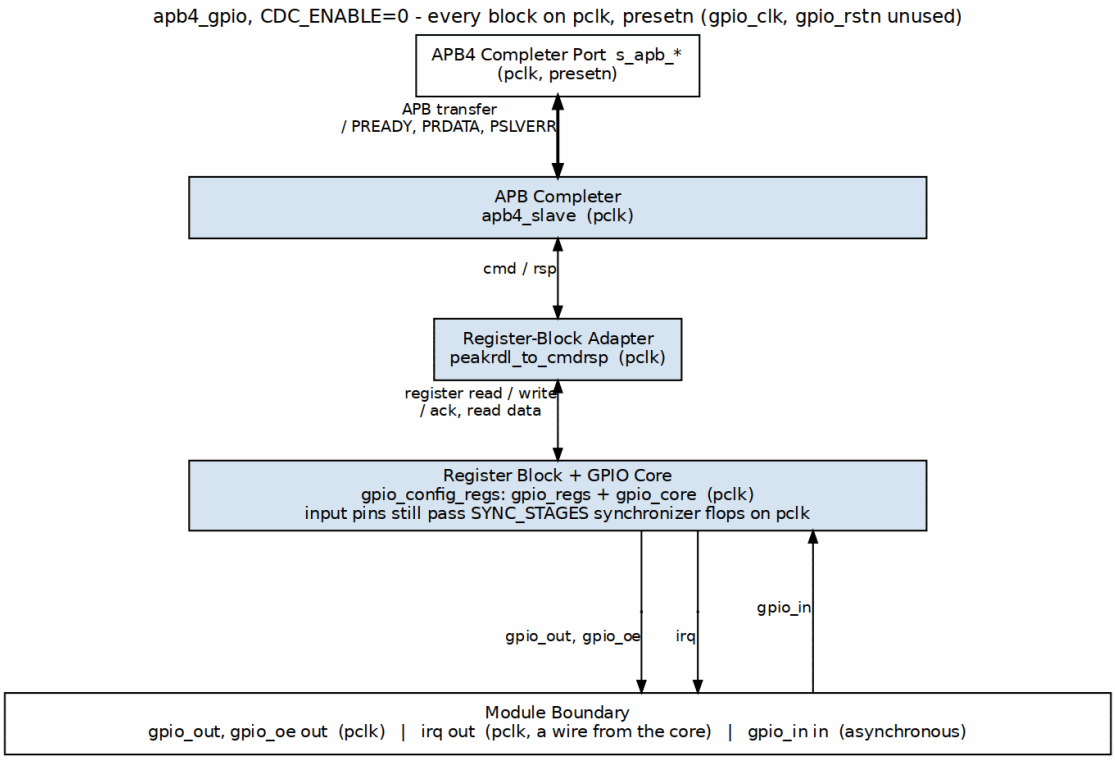
Note: the PeakRDL-generated register file (gpio_regs.sv) is the one exception – it resets SYNCHRONOUSLY (plain always_ff @(posedge clk) with an active-high rst derived from the reset input), so its registers do not reset until a core-clock edge arrives.

Resets: - GPIO output registers - Input synchronizers - Interrupt state

17.2 Functional Description

17.2.1 Clock Configurations

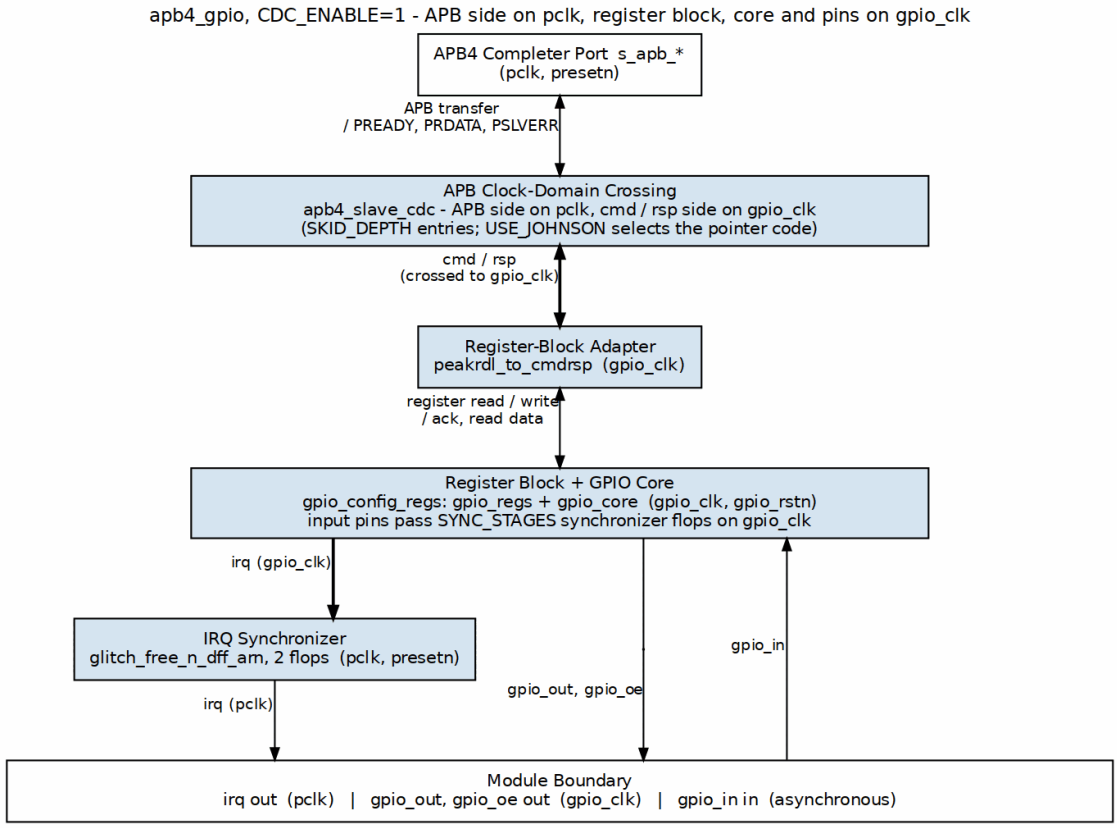
17.2.1.1 Single Clock Domain (CDC_ENABLE = 0)



Single Clock Domain

gpio_clk: Tie to pclk or leave unconnected

17.2.1.2 Dual Clock Domain (CDC_ENABLE = 1)



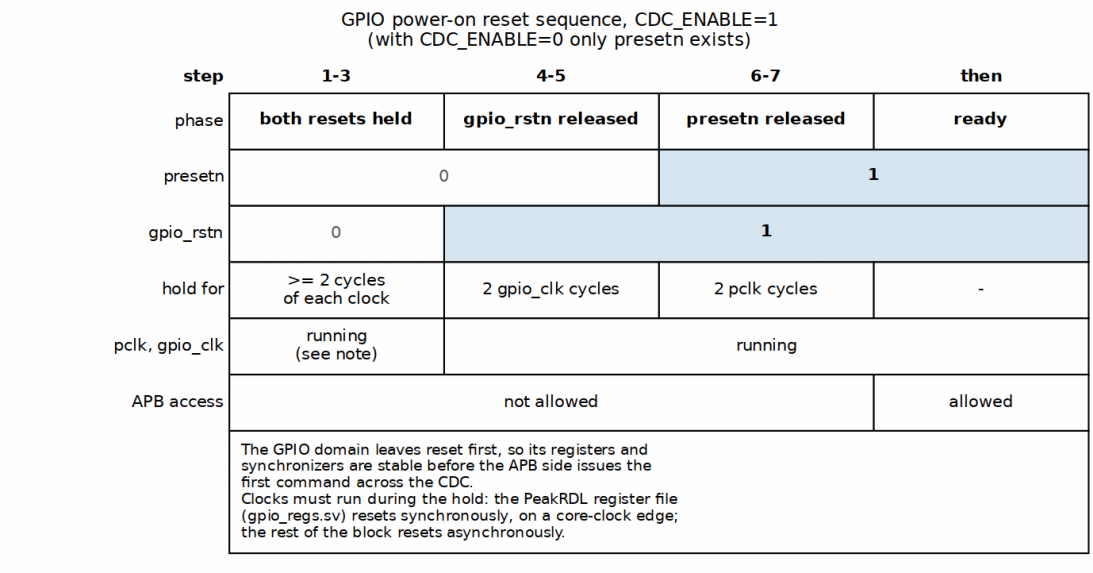
Dual Clock Domain

17.2.2 Reset Sequence

17.2.2.1 Power-On Reset

1. Assert both presetn and gpio_rstn low
2. Keep both clocks running: the PeakRDL register file resets synchronously (see the note above)
3. Hold reset for minimum 2 clock cycles
4. Release gpio_rstn first
5. Wait 2 gpio_clk cycles
6. Release presetn
7. Wait 2 pclk cycles before APB access

17.2.3 Figure 3.2: Power-On Reset Sequence



Power-On Reset Sequence

17.3 Timing

17.3.1 Clock Relationship

Mode	Constraint
CDC_ENABLE=0	Single clock, no special constraints
CDC_ENABLE=1	Set false_path between domains

17.3.2 Reset Recovery

- Allow minimum 2 clock cycles after reset deassert
- First APB transaction may start on 3rd cycle

17.4 Navigation

Back to: 00_overview.md - Interfaces Overview

Next Chapter: Chapter 4: Programming Model

18 APB GPIO - Programming Model Overview

18.1 Overview

Thirteen registers do all the work. Everything in this chapter is a recipe built on these:

Offset	Name	Access	Reset	Description
0x000	GPIO_CONTROL	RW	0x00000001	Global control
0x004	GPIO_DIRECTION	RW	0x00000000	I/O direction
0x008	GPIO_OUTPUT	RW	0x00000000	Output data
0x00C	GPIO_INPUT	RO	-	Input data
0x010	GPIO_INT_ENABLE	RW	0x00000000	Interrupt enable
0x014	GPIO_INT_TYPE	RW	0x00000000	Edge/level select
0x018	GPIO_INT_POLARITY	RW	0xFFFFFFFF	Interrupt polarity
0x01C	GPIO_INT_BOTH	RW	0x00000000	Both edges
0x020	GPIO_INT_STATUS	W1C	0x00000000	Interrupt status
0x024	GPIO_RAW_INT	RO	0 (live)	Raw (unlatched) event status
0x028	GPIO_OUTPUT_SET	WO	0x00000000	Atomic output set (reads return 0)
0x02C	GPIO_OUTPUT_CLR	WO	0x00000000	Atomic output clear (reads return 0)
0x030	GPIO_OUTPUT_TG	WO	0x0000000	Atomic output toggle

Offset	Name	Access	Reset	Description
	L		0	(reads return 0)

18.1.1 In This Chapter

18.1.1.1 Basic Operations

Fundamental GPIO read/write operations.

See: 01_basic_operations.md

18.1.1.2 Interrupt Configuration

Setting up and handling GPIO interrupts.

See: 02_interrupt_config.md

18.1.1.3 Programming Examples

Common use cases with code samples.

See: 03_examples.md

18.1.1.4 Software Considerations

Performance tips and best practices.

See: 04_software_notes.md

18.2 Navigation

Next: 01_basic_operations.md - Basic Operations

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

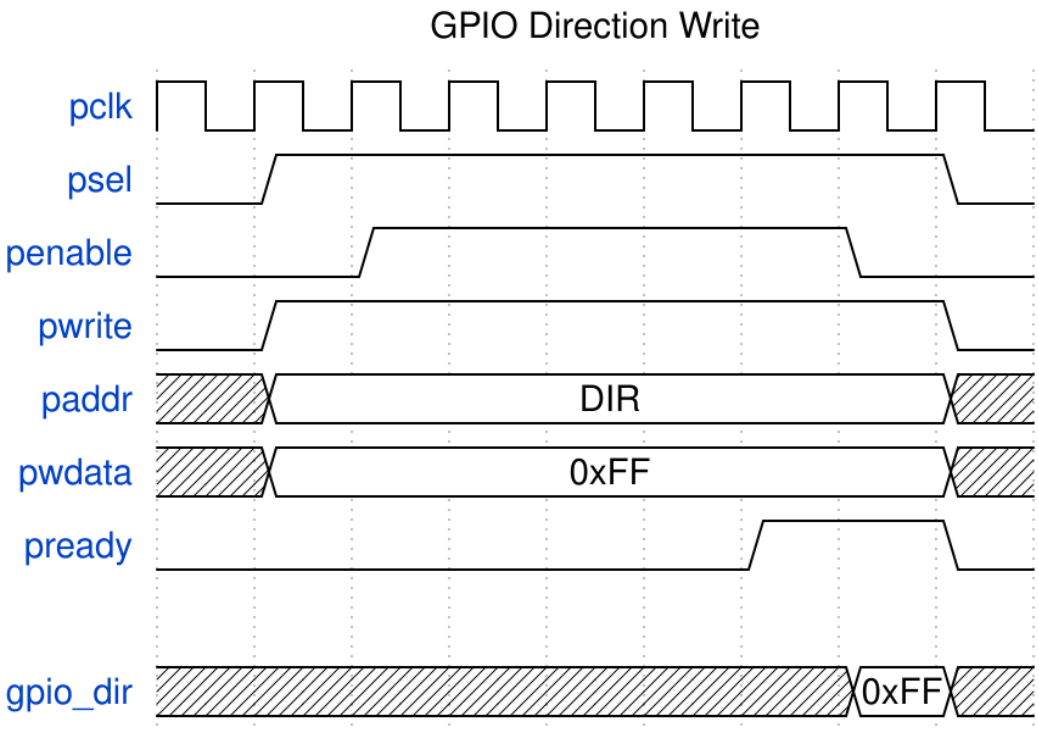
19 APB GPIO - Basic Operations

19.1 Waveforms

Before the code, the timing. These diagrams show what actually happens inside the block for each basic operation.

19.1.1 Direction Configuration

When software writes to GPIO_DIRECTION, the direction register updates and controls the output enable for each pin.

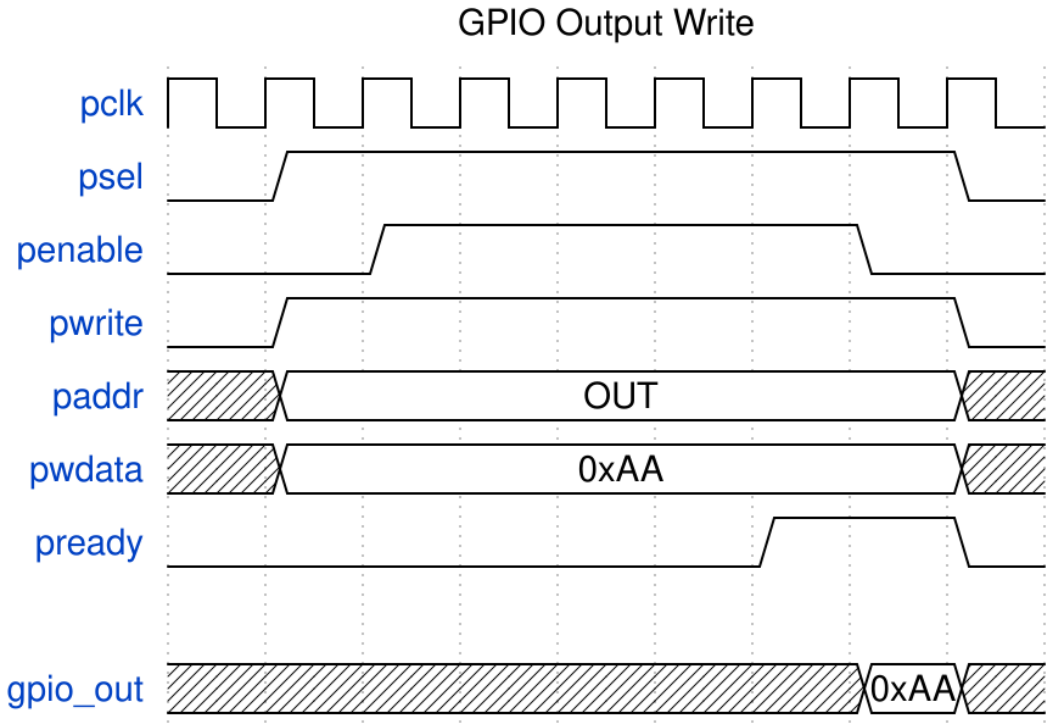


GPIO Direction Write

The APB write is PREADY-gated – a few pclk beyond the two-cycle APB minimum (see Chapter 3.1). The direction register updates on the clock edge following PREADY, and the output enable (`gpio_oe`) reflects the new configuration immediately.

19.1.2 Output Write

Writing to GPIO_OUTPUT sets the output data register, which drives the external pins when direction is set to output.

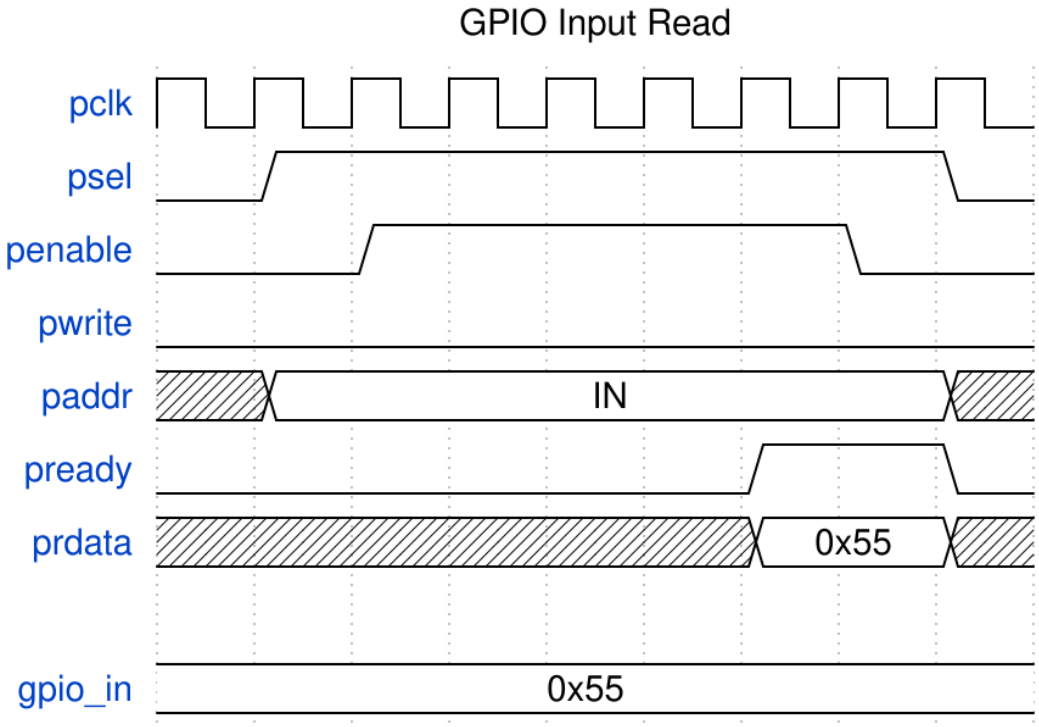


GPIO Output Write

The write data flows through the APB interface to the output register. When `gpio_oe[n]` is high (output mode), `gpio_out[n]` drives the written value to the external pin. The write is strobe-driven, so writing the value the register already holds is still a write and still lands.

19.1.3 Input Read

Reading GPIO_INPUT returns the synchronized input values from external pins.

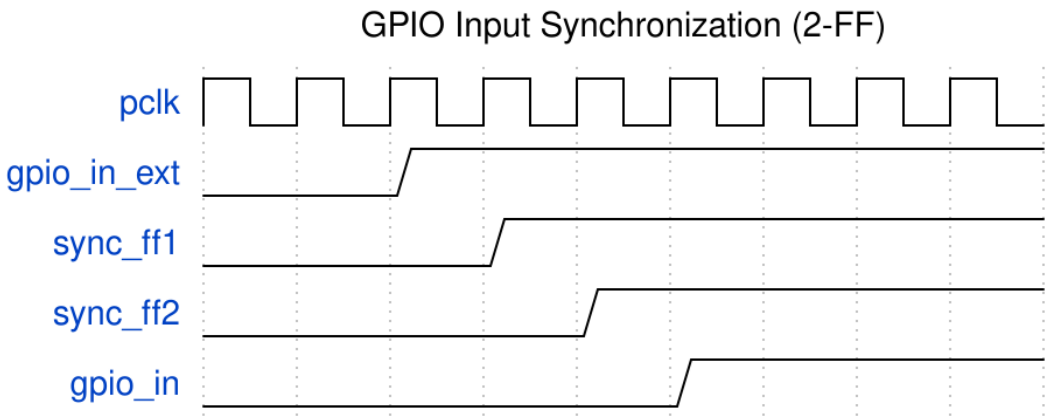


GPIO Input Read

External inputs pass through a 2-stage synchronizer before being captured. The synchronized value (`w_gpio_sync`) is returned on `s_apb_PRDATA` during the APB read transaction.

19.1.4 Input Synchronization

All GPIO inputs pass through a 2-stage synchronizer to prevent metastability.

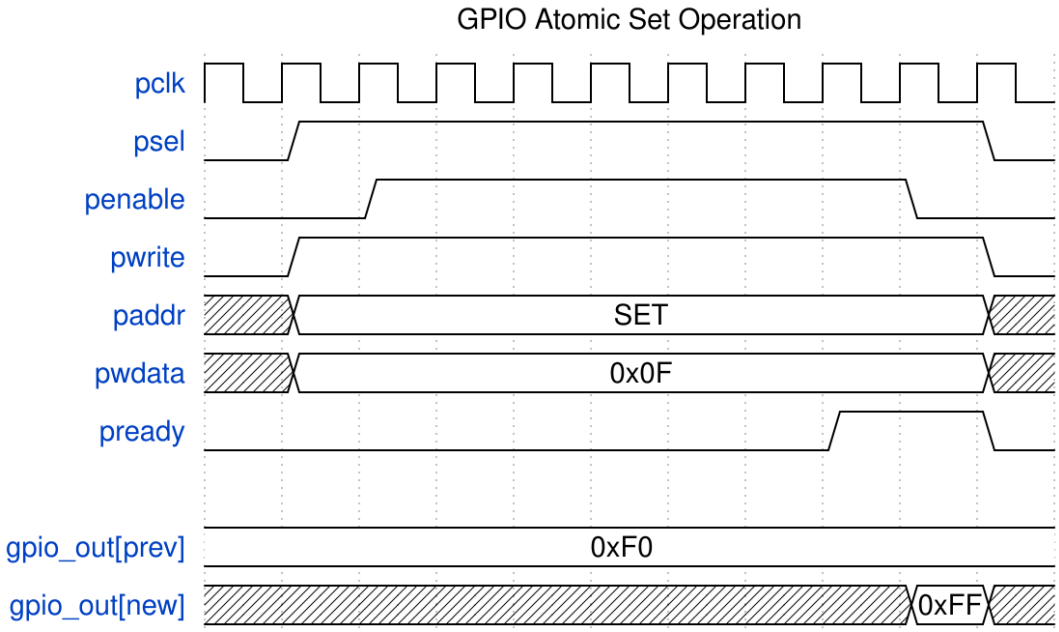


GPIO Input Sync

The synchronizer adds 2 clock cycles of latency. External asynchronous transitions on `gpio_in` propagate through `sync_stage1` and `sync_stage2` before appearing on the internal synchronized signal `w_gpio_sync`.

19.1.5 Atomic Operations

The SET, CLEAR, and TOGGLE registers provide atomic bit manipulation without read-modify-write races.



GPIO Atomic Operations

Three consecutive APB writes demonstrate: 1. **GPIO_OUTPUT_SET (0x028)**: Sets bits where write data is 1, leaves others unchanged 2. **GPIO_OUTPUT_CLR (0x02C)**: Clears bits where write data is 1, leaves others unchanged 3. **GPIO_OUTPUT_TGL (0x030)**: Inverts bits where write data is 1, leaves others unchanged

Each write performs its operation: the RTL fires the atomic pulse from the register's write strobe, so writing the same mask twice performs the operation twice, and nothing needs to be written between operations. The registers are write-only and read as 0; GPIO_OUTPUT reads back the live result. See Chapter 5 for the full semantics.

19.2 Usage Example

19.2.1 Initialization

19.2.1.1 Reset State

After reset: - GPIO_CONTROL = 0x00000001 - output stage ENABLED, global interrupt enable clear - All pins configured as inputs (GPIO_DIRECTION = 0), so every pin is high-Z despite the enable - No per-pin interrupts enabled; GPIO_INT_POLARITY resets to 0xFFFFFFFF (rising/active-high)

19.2.1.2 Enable GPIO

```
// GPIO_CONTROL[0] (ENABLE) already resets to 1; write it explicitly
if a
// previous owner may have cleared it. ENABLE gates only the output
drivers.
GPIO_CONTROL = 0x00000001;
```

19.2.2 Output Operations

19.2.2.1 Configure as Output

```
// Set pins 7:4 as outputs (bits = 1 for output)
GPIO_DIRECTION = 0x000000F0;
```

19.2.2.2 Write Output Values

```
// Set pins 7:4 to value 0101
GPIO_OUTPUT = 0x00000050;
```

19.2.2.3 Toggle Outputs

```
// Atomic toggle of pins 7:4; every write toggles, so a loop that
// writes this each iteration toggles on every pass
GPIO_OUTPUT_TGL = 0x000000F0;
```

19.2.2.4 Atomic Bit Operations

```
// Set specific bits (pins 5 and 7)
GPIO_OUTPUT_SET = 0x000000A0;

// Clear specific bits (pins 4 and 6)
GPIO_OUTPUT_CLR = 0x00000050;
```

Read-modify-write of GPIO_OUTPUT (`GPIO_OUTPUT |= mask`) mixes safely with the atomic registers: GPIO_OUTPUT reads back the live output latch, so after `GPIO_OUTPUT = 0xFF` and `GPIO_OUTPUT_CLR = 0x0F` a read returns 0xF0 and an RMW builds on the real pin state. Prefer the atomic registers where two contexts may update outputs concurrently – that is what they are for.

19.2.3 Input Operations

19.2.3.1 Configure as Input

```
// Set pins 3:0 as inputs (bits = 0 for input)
GPIO_DIRECTION &= ~0x0000000F;
```

19.2.3.2 Read Input Values

```
// Read all inputs
uint32_t inputs = GPIO_INPUT;

// Check specific pin (pin 2)
if (inputs & 0x00000004) {
    // Pin 2 is high
}
```

19.2.3.3 Read with Mask

```
// Read only pins 3:0
uint32_t low_nibble = GPIO_INPUT & 0x0000000F;
```

19.2.4 Mixed I/O Configuration

19.2.4.1 Configure Mixed Directions

```
// Pins 31:16 = outputs, pins 15:0 = inputs
GPIO_DIRECTION = 0xFFFF0000;
```

19.2.4.2 Read-Modify-Write Pattern

```
// Change only pins 11:8 to outputs
uint32_t dir = GPIO_DIRECTION;
dir |= 0x00000F00;    // Set pins 11:8
GPIO_DIRECTION = dir;
```

19.2.5 Output Enable Behavior

19.2.5.1 Hardware Interface

When direction bit is set: - `gpio_oe[i]` = 1 (output enabled) - `gpio_out[i]` = GPIO_OUTPUT[i] value

When direction bit is clear: - `gpio_oe[i]` = 0 (high impedance) - `gpio_out[i]` = don't care

19.2.5.2 Glitch Considerations

To avoid output glitches when switching direction: 1. Set GPIO_OUTPUT to desired value 2. Then change GPIO_DIRECTION

```
// Safe direction change to output
GPIO_OUTPUT = desired_value;    // Set value first
GPIO_DIRECTION |= pin_mask;     // Then enable output
```

19.3 Navigation

Next: 02_interrupt_config.md - Interrupt Configuration

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

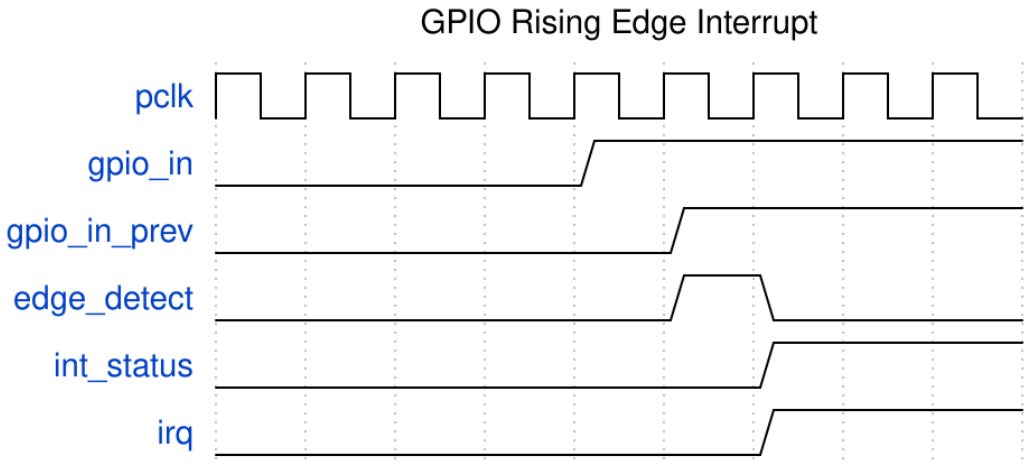
20 APB GPIO - Interrupt Configuration

20.1 Waveforms

First, what the hardware does. These diagrams walk through detection and clearing, one mode at a time.

20.1.1 Rising Edge Interrupt

Edge-triggered interrupts detect transitions on input pins.

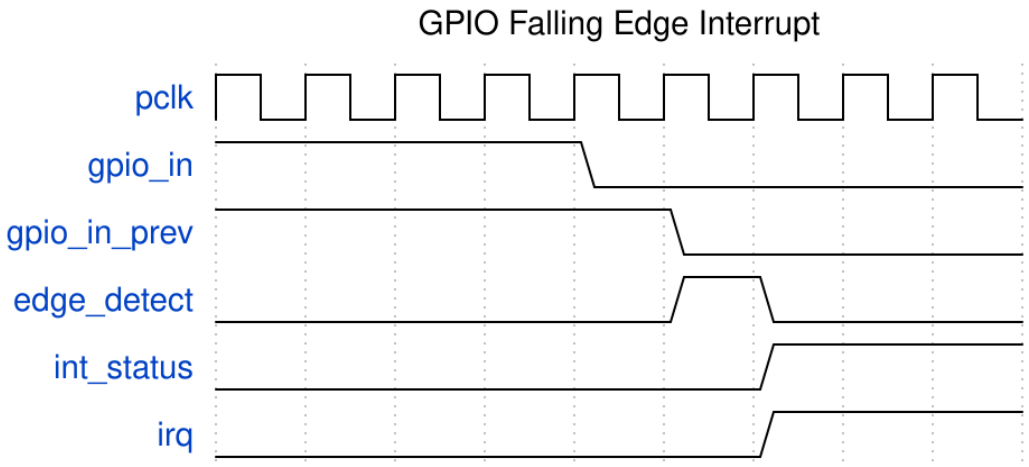


GPIO Rising Edge Interrupt

The detection sequence: 1. External input `gpio_in[0]` transitions from 0 to 1 2. 2-stage synchronizer captures the transition (`w_gpio_sync`) 3. Edge detector compares current vs. delayed value (`r_gpio_prev`) 4. Rising edge pulse (`w_rising_edge`) generated for one clock 5. Raw pulse (`w_raw_int[0]`, combinational) sets the sticky GPIO_INT_STATUS bit in the register file 6. Combined `irq` output asserts

20.1.2 Falling Edge Interrupt

Falling edge detection uses inverted polarity configuration.

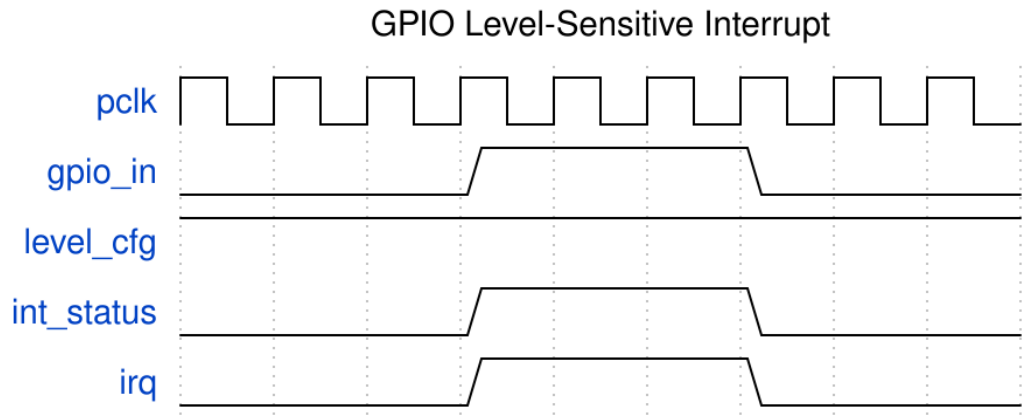


GPIO Falling Edge Interrupt

With `cfg_int_type[0]=0` (edge mode) and `cfg_int_polarity[0]=0` (falling edge), the detector triggers on 1-to-0 transitions.

20.1.3 Level-Sensitive Interrupt

Level-sensitive interrupts track the input state directly.

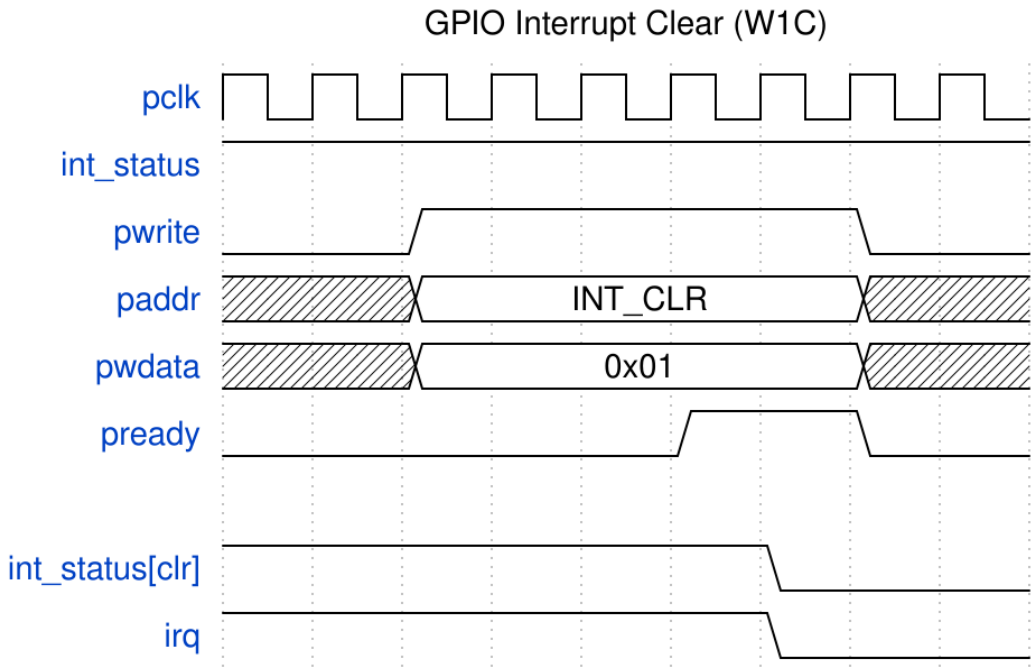


GPIO Level Interrupt

Key differences from edge mode: - `irq` follows the input level (not latched) - No edge detection logic involved - Interrupt re-asserts if source not cleared before ISR exit

20.1.4 Interrupt Clear (W1C)

Write-1-to-Clear mechanism clears latched interrupts.



GPIO Interrupt Clear

The clear sequence: 1. GPIO_INT_STATUS[0] is set (edge was detected and latched) 2. Software writes 0x01 to INT_STATUS register 3. W1C logic clears the sticky status bit 4. `irq` deasserts

Note: For level-sensitive interrupts, the external source must be cleared first, otherwise the interrupt immediately re-asserts.

20.2 Functional Description

20.2.1 Interrupt Configuration Table

Every combination of the three mode fields, in one place:

INT_TYPE	INT_POLARITY	INT_BOTH	Trigger
0	0	0	Falling edge
0	1	0	Rising edge
0	X	1	Both edges
1	0	X	Active low
1	1	X	Active high

20.3 Usage Example

20.3.1 Interrupt Setup

Four steps, in order. Skipping step 4 is the classic mistake — the global enable resets to 0, so nothing ever interrupts.

20.3.1.1 1. Configure Interrupt Type

```
// Edge-triggered (0) or Level-sensitive (1)
// Pins 3:0 edge, pins 7:4 level
GPIO_INT_TYPE = 0x000000F0;
```

20.3.1.2 2. Configure Polarity

For edge mode: - 0 = falling edge - 1 = rising edge

For level mode: - 0 = active low - 1 = active high

```
// Rising edge / active high for pins 7:0
GPIO_INT_POLARITY = 0x000000FF;
```

20.3.1.3 3. Configure Both-Edge Mode (Edge Mode Only)

```
// Enable both edges for pin 0
GPIO_INT_BOTH = 0x00000001;
```

20.3.1.4 4. Enable Interrupts

```
// Enable interrupts on pins 7:0
GPIO_INT_ENABLE = 0x000000FF;

// Set the GLOBAL interrupt enable, GPIO_CONTROL[1] - it resets to 0
and
// gates the irq output, so without this step no interrupt ever
asserts.
GPIO_CONTROL |= 0x00000002;
```

20.3.2 Complete Setup Examples

20.3.2.1 Rising Edge Interrupt

```
// Configure pin 5 for rising edge interrupt
GPIO_INT_TYPE &= ~(1 << 5);      // Edge mode
GPIO_INT_POLARITY |= (1 << 5);   // Rising edge
GPIO_INT_BOTH &= ~(1 << 5);      // Single edge
GPIO_INT_ENABLE |= (1 << 5);     // Per-pin enable
GPIO_CONTROL |= 0x2;             // Global INT_ENABLE (resets to 0;
irq never asserts without it)
```

20.3.2.2 Both-Edge Interrupt

```
// Configure pin 3 for both-edge interrupt
GPIO_INT_TYPE &= ~(1 << 3);      // Edge mode
GPIO_INT_BOTH |= (1 << 3);       // Both edges
GPIO_INT_ENABLE |= (1 << 3);     // Per-pin enable
GPIO_CONTROL |= 0x2;             // Global INT_ENABLE
```

20.3.2.3 Active-Low Level Interrupt

```
// Configure pin 7 for active-low level interrupt
GPIO_INT_TYPE |= (1 << 7);       // Level mode
GPIO_INT_POLARITY &= ~(1 << 7);  // Active low
GPIO_INT_ENABLE |= (1 << 7);     // Per-pin enable
GPIO_CONTROL |= 0x2;             // Global INT_ENABLE
```

20.3.3 Interrupt Handling

20.3.3.1 Check Interrupt Status

```
uint32_t status = GPIO_INT_STATUS;
```

20.3.3.2 Clear Interrupts (Write-1-to-Clear)

```
// Clear specific interrupt (pin 5)
GPIO_INT_STATUS = (1 << 5);

// Clear all pending interrupts
GPIO_INT_STATUS = 0xFFFFFFFF;
```

20.3.3.3 Complete ISR Example

```
void gpio_isr(void) {
    // Read status
    uint32_t status = GPIO_INT_STATUS;

    // Handle each pending interrupt
    for (int i = 0; i < 32; i++) {
        if (status & (1 << i)) {
            handle_gpio_event(i);
        }
    }

    // Clear handled interrupts
    GPIO_INT_STATUS = status;
}
```

20.3.4 Level-Sensitive Considerations

20.3.4.1 Avoid Interrupt Storm

For level-sensitive interrupts, the source must be cleared before the status:

```
void level_sensitive_isr(void) {
    uint32_t status = GPIO_INT_STATUS;

    // For level-sensitive pins, handle source first
    if (status & LEVEL_PIN_MASK) {
        clear_external_source();  // Clear what's driving pin
    }

    // Then clear status
    GPIO_INT_STATUS = status;
}
```

20.3.4.2 Masking During Handling

Use the global enable, GPIO_CONTROL[1]. Clearing GPIO_INT_ENABLE does NOT mask an edge interrupt that has already latched into GPIO_INT_STATUS (the per-pin enable is only sampled when the event sets the sticky bit), so an enable-save/restore recipe leaves `irq` asserted for pending edge sources.

```
// Temporarily mask everything while handling
GPIO_CONTROL &= ~0x00000002;   // global interrupt enable off

// Handle interrupt source
handle_interrupt();

// Unmask
GPIO_CONTROL |= 0x00000002;
```

20.4 Navigation

Next: 03_examples.md - Programming Examples

---

21 APB GPIO - Programming Examples

21.1 Usage Example

Real recipes for the things people actually hang off a GPIO. Copy, paste, adjust the pin numbers.

21.1.1 LED Control

21.1.1.1 Simple LED Blink

```
#define LED_PIN  (1 << 0)

void led_init(void) {
    GPIO_CONTROL = 1;            // Enable GPIO
    GPIO_DIRECTION |= LED_PIN;   // Set as output
}

void led_on(void) {
    GPIO_OUTPUT |= LED_PIN;
}

void led_off(void) {
    GPIO_OUTPUT &= ~LED_PIN;
}

void led_toggle(void) {
    GPIO_OUTPUT ^= LED_PIN;
}
```

The read-modify-write style above is fine alongside the atomic GPIO_OUTPUT_SET/CLR/TGL registers: GPIO_OUTPUT reads back the live output latch, so an RMW always starts from what the pins are actually driving (Chapter 5). Where two contexts may update outputs at once, the atomic registers avoid the RMW race entirely, and every write takes effect:

```
void led_on(void)     { GPIO_OUTPUT_SET = LED_PIN; }
void led_off(void)    { GPIO_OUTPUT_CLR = LED_PIN; }
void led_toggle(void) { GPIO_OUTPUT_TGL = LED_PIN; }
```

21.1.1.2 Multiple LED Control

```
#define LED_MASK  0x000000FF  // LEDs on pins 7:0

void leds_write(uint8_t pattern) {
    uint32_t output = GPIO_OUTPUT;
    output = (output & ~LED_MASK) | pattern;
    GPIO_OUTPUT = output;
}
```

21.1.2 Button Input

21.1.2.1 Simple Button Read

```
#define BUTTON_PIN  (1 << 8)

void button_init(void) {
    GPIO_CONTROL = 1;              // Enable GPIO
    GPIO_DIRECTION &= ~BUTTON_PIN; // Set as input
}

bool button_pressed(void) {
    return (GPIO_INPUT & BUTTON_PIN) != 0;
}
```

21.1.2.2 Button with Interrupt

```
#define BUTTON_PIN  (1 << 8)

volatile bool button_event = false;

void button_init_irq(void) {
    GPIO_CONTROL = 1;
    GPIO_DIRECTION &= ~BUTTON_PIN;

    // Configure falling edge interrupt (button press)
    GPIO_INT_TYPE &= ~BUTTON_PIN;      // Edge mode
    GPIO_INT_POLARITY &= ~BUTTON_PIN;  // Falling edge
    GPIO_INT_BOTH &= ~BUTTON_PIN;      // Single edge
    GPIO_INT_ENABLE |= BUTTON_PIN;     // Enable pin
    GPIO_CONTROL |= 0x2;               // Global interrupt enable
(resets to 0)
}

void button_isr(void) {
    if (GPIO_INT_STATUS & BUTTON_PIN) {
        button_event = true;
        GPIO_INT_STATUS = BUTTON_PIN;  // Clear
    }
}
```

21.1.3 DIP Switch Reading

21.1.3.1 8-Bit Switch Input

```
#define SWITCH_MASK  0x00FF0000  // Switches on pins 23:16
#define SWITCH_SHIFT 16

void switch_init(void) {
```

```
    GPIO_CONTROL = 1;
    GPIO_DIRECTION &= ~SWITCH_MASK;  // All inputs
}

uint8_t switch_read(void) {
    return (GPIO_INPUT & SWITCH_MASK) >> SWITCH_SHIFT;
}
```

21.1.4 Parallel Data Interface

21.1.4.1 8-Bit Output Port

```
#define DATA_MASK   0x000000FF  // Data on pins 7:0
#define STROBE_PIN  (1 << 8)    // Strobe on pin 8

void data_port_init(void) {
    GPIO_CONTROL = 1;
    GPIO_DIRECTION |= (DATA_MASK | STROBE_PIN);
    GPIO_OUTPUT &= ~STROBE_PIN;  // Strobe low
}

void data_write(uint8_t data) {
    uint32_t output = GPIO_OUTPUT;
    output = (output & ~DATA_MASK) | data;
    GPIO_OUTPUT = output;

    // Generate strobe pulse
    GPIO_OUTPUT |= STROBE_PIN;
    // Small delay if needed
    GPIO_OUTPUT &= ~STROBE_PIN;
}
```

21.1.4.2 8-Bit Input Port with Ready

```
#define DATA_MASK   0x000000FF  // Data on pins 7:0
#define READY_PIN   (1 << 8)    // Ready on pin 8

void data_input_init(void) {
    GPIO_CONTROL = 1;
    GPIO_DIRECTION &= ~(DATA_MASK | READY_PIN);

    // Interrupt on ready rising edge
    GPIO_INT_TYPE &= ~READY_PIN;
    GPIO_INT_POLARITY |= READY_PIN;
    GPIO_INT_ENABLE |= READY_PIN;
}

uint8_t data_read(void) {
```

```
    return GPIO_INPUT & DATA_MASK;
}
```

21.1.5 PWM-Style Output

21.1.5.1 Bit-Banged PWM (Low Frequency)

```
#define PWM_PIN  (1 << 0)

void pwm_init(void) {
    GPIO_CONTROL = 1;
    GPIO_DIRECTION |= PWM_PIN;
}

// Call from timer interrupt
void pwm_update(uint8_t duty, uint8_t *counter) {
    (*counter)++;
    if (*counter >= 100) *counter = 0;

    if (*counter < duty) {
        GPIO_OUTPUT |= PWM_PIN;
    } else {
        GPIO_OUTPUT &= ~PWM_PIN;
    }
}
```

21.1.6 Wake-On-Change

21.1.6.1 Power Management Integration

```
#define WAKE_PINS  0x0000000F  // Wake sources on pins 3:0

void wake_setup(void) {
    // Configure both-edge interrupts for wake pins
    GPIO_INT_TYPE &= ~WAKE_PINS;   // Edge mode
    GPIO_INT_BOTH |= WAKE_PINS;    // Both edges
    GPIO_INT_ENABLE |= WAKE_PINS;  // Enable

    // Clear any pending before sleep
    GPIO_INT_STATUS = WAKE_PINS;
}

void enter_sleep(void) {
    wake_setup();
    // Platform-specific sleep entry
    __WFI();  // Wait for interrupt
}
```

21.2 Navigation

Next: 04_software_notes.md - Software Considerations

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

22 APB GPIO - Software Considerations

22.1 Design Notes

The stuff that separates a driver that works from a driver that works reliably. None of this is in the register map; all of it matters.

22.1.1 Performance

22.1.1.1 Register Access Timing

Operation	APB Cycles	Notes
Read	2 + wait states	PREADY-gated (bridge FSM + cmd/rsp handshake)
Write	2 + wait states	PREADY-gated
Read-Modify-Write	one read + one write	Each PREADY-gated

22.1.1.2 Optimizing Access

Batch operations when possible:

```
// Inefficient - 4 separate writes
GPIO_OUTPUT |= (1 << 0);
GPIO_OUTPUT |= (1 << 1);
GPIO_OUTPUT |= (1 << 2);
GPIO_OUTPUT |= (1 << 3);

// Efficient - single write
GPIO_OUTPUT |= 0x0000000F;
```

Cache register values:

```
// Inefficient - 4 reads + 4 writes
for (int i = 0; i < 4; i++) {
    GPIO_DIRECTION |= (1 << i);
}

// Efficient - 1 read + 1 write
uint32_t dir = GPIO_DIRECTION;
dir |= 0x0000000F;
GPIO_DIRECTION = dir;
```

22.1.2 Synchronization

22.1.2.1 Input Latency

GPIO inputs have inherent latency: - SYNC_STAGES clock cycles (default 2) - Plus software polling/interrupt overhead

Account for latency in timing-critical code.

22.1.2.2 Volatile Registers

Always declare GPIO registers as volatile:

```
#define GPIO_INPUT  (*(volatile uint32_t *)0xFEC0700C)
```

22.1.2.3 Multi-Core Considerations

If multiple cores access GPIO:

```
// Use atomic operations or locks
spin_lock(&gpio_lock);
uint32_t val = GPIO_OUTPUT;
val |= new_bits;
GPIO_OUTPUT = val;
spin_unlock(&gpio_lock);
```

22.1.3 Interrupt Best Practices

22.1.3.1 Clear Before Return

Always clear interrupt status before ISR return:

```
void gpio_isr(void) {
    uint32_t status = GPIO_INT_STATUS;
    // Handle interrupts
    GPIO_INT_STATUS = status;  // Must clear!
}
```

22.1.3.2 Avoid Spurious Interrupts

Disable interrupts during configuration:

```
void reconfigure_interrupt(int pin) {
    uint32_t mask = (1 << pin);

    // Disable first
    GPIO_INT_ENABLE &= ~mask;

    // Reconfigure
    // ...
```

```
    // Clear any pending
    GPIO_INT_STATUS = mask;

    // Re-enable
    GPIO_INT_ENABLE |= mask;
}
```

22.1.3.3 Level-Sensitive Caution

Level interrupts can cause interrupt storms:

```
void level_isr(void) {
    // WRONG - will re-trigger immediately
    GPIO_INT_STATUS = status;

    // RIGHT - handle source first
    clear_external_interrupt_source();
    GPIO_INT_STATUS = status;
}
```

22.1.4 Error Handling

22.1.4.1 No Hardware Errors

GPIO controller doesn't generate errors: - All addresses valid - pslverr always 0

22.1.4.2 Software Validation

Validate configuration in software:

```
bool gpio_set_direction(uint32_t pin, bool output) {
    if (pin >= 32) return false;

    if (output) {
        GPIO_DIRECTION |= (1 << pin);
    } else {
        GPIO_DIRECTION &= ~(1 << pin);
    }
    return true;
}
```

22.1.5 Debug Tips

22.1.5.1 Read-Back Verification

```
void gpio_debug(void) {
    printf("CONTROL:   0x%08X\n", GPIO_CONTROL);
    printf("DIRECTION: 0x%08X\n", GPIO_DIRECTION);
    printf("OUTPUT:    0x%08X\n", GPIO_OUTPUT);
```

```
    printf("INPUT:     0x%08X\n", GPIO_INPUT);
    printf("INT_STAT:  0x%08X\n", GPIO_INT_STATUS);
}
```

22.1.5.2 Loopback Testing

Connect output to input for self-test:

```
bool gpio_loopback_test(int out_pin, int in_pin) {
    GPIO_DIRECTION |= (1 << out_pin);
    GPIO_DIRECTION &= ~(1 << in_pin);

    // Test high
    GPIO_OUTPUT |= (1 << out_pin);
    delay_us(10);  // Allow synchronization
    if (!(GPIO_INPUT & (1 << in_pin))) return false;

    // Test low
    GPIO_OUTPUT &= ~(1 << out_pin);
    delay_us(10);
    if (GPIO_INPUT & (1 << in_pin)) return false;

    return true;
}
```

22.2 Navigation

Back to: 00_overview.md - Programming Model Overview

Next Chapter: Chapter 5: Registers

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

23 APB GPIO - Register Map

23.1 Overview

Thirteen registers, verified against the RTL decode (`gpio_regs.sv`) and `../../rdl/gpio/gpio_regs.rdl`. `GPIO_REGS_SIZE = 0x34`.

Offset	Name	Access	Reset	Description
0x000	GPIO_CONTROL	RW	0x00000001	Global enable + global interrupt enable
0x004	GPIO_DIRECTION	RW	0x00000000	Pin direction
0x008	GPIO_OUTPUT	RW	0x00000000	Output data
0x00C	GPIO_INPUT	RO	-	Input data
0x010	GPIO_INT_ENABLE	RW	0x00000000	Per-pin interrupt enable
0x014	GPIO_INT_TYPE	RW	0x00000000	Interrupt type
0x018	GPIO_INT_POLARITY	RW	0xFFFFFFFF	Interrupt polarity
0x01C	GPIO_INT_BOTH	RW	0x00000000	Both-edge enable
0x020	GPIO_INT_STATUS	W1C	0x00000000	Latched interrupt status
0x024	GPIO_RAW_INT	RO	0 (live)	Live (unlatched) event detector output – reads 0 at reset only because type/polarity/sync all reset to 0; depends on pins thereafter
0x028	GPIO_OUTPUT_SET	WO	0x00000000	Atomic output set (reads return 0)

Offset	Name	Access	Reset	Description
0x02C	GPIO_OUTPUT_CLR	WO	0x00000000	Atomic output clear (reads return 0)
0x030	GPIO_OUTPUT_TGL	WO	0x00000000	Atomic output toggle (reads return 0)

Address decode and aliasing: only address bits [5:0] reach the register block (`gpio_config_regs.sv` passes `regblk_addr[5:0]`), so the 13-register map aliases every 64 bytes across the 4KB APB window. Offsets 0x034-0x03F within each 64-byte tile are unmapped and read as zero; no access ever raises PSLVERR.

23.2 GPIO_CONTROL (0x000)

Global control register.

Bits	Name	Access	Reset	Description
31:2	Reserved	RO	0	Reserved
1	INT_ENABLE	RW	0	Global interrupt enable. Gates the `irq` output: `irq = INT_ENABLE && (any effective pending interrupt)`. Resets to 0, so no interrupt asserts until software sets this bit.
0	ENABLE	RW	1	GPIO output enable. Gates only `gpio_oe`: when 0, every pin is high-Z regardless of GPIO_DIRECTION. Output data, input synchronization, and interrupt detection are unaffected. Resets to 1 (enabled).

23.3 GPIO_DIRECTION (0x004)

Pin direction control. Each bit controls one GPIO pin.

Bits	Name	Access	Reset	Description
31:0	DIR	RW	0	Direction per pin (0=input, 1=output)

23.4 GPIO_OUTPUT (0x008)

Output data register. Values driven when pin configured as output.

Bits	Name	Access	Reset	Description
31:0	DATA	RW	0	Output values per pin

Readback: reads return the live output latch (`r_output_data` in `gpio_core.sv`), not merely the last value software wrote. After any atomic operation (GPIO_OUTPUT_SET/CLR/TGL) the core writes its latch back into this register one cycle later, so the register and the pins stay in agreement: write 0xFF here, then 0x0F to GPIO_OUTPUT_CLR, and a read returns 0xF0. A read-modify-write of GPIO_OUTPUT therefore mixes safely with the atomic registers. The write itself is strobe-driven, so writing the value the register already holds still lands, and a software write wins over a coincident hardware write-back.

23.5 GPIO_INPUT (0x00C)

Input data register. Reflects synchronized external pin values.

Bits	Name	Access	Reset	Description
31:0	DATA	RO	-	Input values per pin

Note: Value depends on external signals, not reset.

23.6 GPIO_INT_ENABLE (0x010)

Interrupt enable register. Controls which pins can generate interrupts.

Bits	Name	Access	Reset	Description
31:0	IE	RW	0	Interrupt enable per pin (1=enabled)

23.7 GPIO_INT_TYPE (0x014)

Interrupt type select. Chooses edge or level sensitivity.

Bits	Name	Access	Reset	Description
31:0	TYPE	RW	0	Type per pin (0=edge, 1=level)

23.8 GPIO_INT_POLARITY (0x018)

Interrupt polarity select.

Bits	Name	Access	Reset	Description
31:0	POL	RW	0xFFFFFFFF	Polarity per pin (resets to all rising/active-high)

For edge mode: 0=falling, 1=rising For level mode: 0=active-low, 1=active-high

23.9 GPIO_INT_BOTH (0x01C)

Both-edge interrupt enable. Only applicable in edge mode.

Bits	Name	Access	Reset	Description
31:0	BOTH	RW	0	Both edges per pin (1=both edges)

When set, GPIO_INT_POLARITY is ignored for that pin.

23.10 GPIO_INT_STATUS (0x020)

Latched interrupt status register.

Bits	Name	Access	Reset	Description
31:0	STATUS	W1C	0	Latched interrupt event per pin

Access: Read returns current status. Write 1 clears the bit.

Semantics (per the RTL):

- A bit sets only when the event occurs while that pin's GPIO_INT_ENABLE bit is 1 (`sts_int_pending = raw & enable`, `gpio_core.sv`). Events on disabled pins set nothing.
- Bits latch (stay set) until written-1-to-clear, in both edge and level modes. In level mode the bit remains set after the input level clears.
- The `irq` output uses this register only for edge-mode pins. Level-mode pins drive `irq` from the live detector output, so W1C on a level pin does not deassert `irq` while the level persists (see Chapter 3.3).
- A W1C write that lands in the same cycle as a new hardware event is merged per bit, `next = (value | hw_set) & ~w1c_mask`: bits the write is not clearing keep the coincident event (`gpio_config_regs.sv` holds the set in a one-cycle deferred-set register and applies it once the write completes), and a bit the write IS clearing is cleared, as in any W1C register. No event is lost on a bit software did not ask to clear.

23.11 GPIO_RAW_INT (0x024)

Live event-detector output, before per-pin enable gating and latching.

Bits	Name	Access	Reset	Description
31:0	RAW	RO	0	Raw interrupt condition per pin

Reflects the edge/level detector output combinationally; not affected by GPIO_INT_ENABLE, GPIO_CONTROL, or W1C.

23.12 GPIO_OUTPUT_SET (0x028)

Atomic output set: writing 1 to a bit sets the corresponding output pin without a read-modify-write of GPIO_OUTPUT.

Bits	Name	Access	Reset	Description
31:0	SET	WO	0	Set output pins (1=set); reads return 0

23.13 GPIO_OUTPUT_CLR (0x02C)

Atomic output clear.

Bits	Name	Access	Reset	Description
31:0	CLR	WO	0	Clear output pins (1=clear); reads return 0

23.14 GPIO_OUTPUT_TGL (0x030)

Atomic output toggle.

Bits	Name	Access	Reset	Description
31:0	TGL	WO	0	Toggle output pins (1=toggle); reads return 0

23.15 Design Notes

23.15.1 Atomic-Register Write Semantics (SET/CLR/TGL)

Every write performs the operation, driven by the register's write strobe (`gpio_config_regs.sv`). The regblock exports `swmod` for each register; it is a level held for the whole command-bridge transaction and it leads the field storage by one cycle, so the wrapper takes its rising edge (one write, one event), delays it one flop to line up with the stored mask, and gates the mask with that one-cycle strobe to form the pulse gpio_core acts on:

```
assign w_set_wr_event = w_set_swmod & ~r_set_swmod_d;   // rising edge
r_set_stb <= w_set_wr_event;                            // align to
field
assign w_output_set = r_set_stb ? GPIO_OUTPUT_SET.set_bits.value : '0;
```

SystemRDL restricts `singlepulse` to 1-bit fields, which is why the 32-bit masks are pulsed this way rather than self-clearing.

What software can rely on:

- Writing the same mask twice performs the operation twice. A toggle loop writing `GPIO_OUTPUT_TGL = mask` each iteration toggles on every pass.
- Alternating SET and CLR of the same mask performs every operation; nothing needs to be written in between, and no software shadow of the mask is needed.
- The registers are write-only by design (the RDL declares sw = w): reads at 0x028/0x02C/0x030 return 0. The result of an operation is observable in GPIO_OUTPUT, which reads back the live latch.

These are the conventional set/clear/toggle semantics (fixed 2026-09-08, issue #44).

23.15.2 Address Calculation

For system address:

```
Register_Address = BASE_ADDR + WINDOW_OFFSET + Register_Offset

Where:
  BASE_ADDR = 0xFEC00000 (RLB base)
  WINDOW_OFFSET = 0x7000 (GPIO window)
  Register_Offset = value from table above

Example:
  GPIO_INPUT = 0xFEC00000 + 0x7000 + 0x00C = 0xFEC0700C
```

23.16 Navigation

Back to: GPIO Specification Index